Aalto-yliopisto
Aalto-universitetet
Aalto University

Master's Programme in Building Technology

Multi-Agent Retrieval-Augmented System for Domain-Specific Knowledge in Structural Engineering

Hafiz Muhammad Ahmad

Master's thesis
2025

Copyright ©2025 Hafiz Muhammad Ahmad

Aalto-yliopisto
Aalto-universitetet
Aalto University

Author Hafiz Muhammad Ahmad

Title of thesis Multi-Agent Retrieval-Augmented System for Domain-Specific Knowledge in Structural Engineering

Programme Building Technology

Major Building Technology

Thesis supervisor Assistant Prof. Mohamed Noureldin

Thesis advisor Petri Kortelainen

Collaborative partner Ramboll

Date 29.09.2025 **Number of pages** 65 **Language** English

Abstract

Large Language Models (LLMs) have shown remarkable capabilities in natural language tasks, but they struggle with specialized engineering knowledge like country-specific code provisions. This thesis addresses that gap by integrating the Finnish National Annexes to the Eurocode into an LLM-based assistant. We design a multi-agent Retrieval-Augmented Generation (RAG) system with two agents: one dedicated to retrieving relevant text from Finnish annex documents, and a second that leverages general Eurocode knowledge if the first agent finds no matches. We evaluate the system on structural engineering queries drawn from real design scenarios, comparing different model settings. Our results show that the RAG approach produces accurate, code-compliant answers grounded in the annex text, while the two-agent design guarantees a response for every query. Key findings indicate that smaller LLM variants stay strictly within provided sources by eliminating hallucinations at the cost of leaving some questions partially answered, whereas larger models offer more complete answers by drawing on general knowledge but occasionally introduce unsupported details. The thesis contributions include a novel LLM-based tool for automated code compliance assistance, empirical insights into model behavior with RAG, and a step toward safer AI integration in structural design workflows.

Keywords Generative AI, LLMs, Agentic AI, RAG, Structural Engineering

Table of contents

Preface and acknowledgements 6
Symbols and abbreviations 7
Abbreviations 7
1 Introduction 8
1.1 Research Objectives and Questions 9
1.2 Structure of the Thesis 10
2 Literature review 11
2.1 Large Language Models (LLMs) in Engineering Applications 11
2.1.1 Fundamentals of LLMs 11
2.1.2 History of LLMs 12
2.1.3 Capabilities and Limitations for Engineering Tasks 13
2.1.4 Existing research of LLMs in structural engineering 15
2.2 AI Agents and Retrieval-Augmented Generation (RAG) in Design Workflows 16
2.2.1 Theoretical foundations of Retrieval-Augmented Generation (RAG) 17
2.2.2 Introduction to AI Agents and Agentic AI 19
2.2.3 Role of RAG and Agentic RAG in Improving AI Decision-Making 21
2.3 AI in Structural Engineering 22
2.3.1 Current Trends in AI for Structural Engineering 22
2.3.2 Challenges in Generative AI for Structural Design 23
2.4 Training AI for Structural Design Code Compliance 24
2.4.1 Overview of Structural Design Codes (Eurocodes and National Finnish Annexes) 24
2.4.2 Training LLMs to Understand and Apply Codes 25
2.4.3 The Role of Explainability and Interpretability 26
3 Research material and methods 27
3.1 Introduction to Methodological Framework 27
3.1.1 Research Methodology Overview 27
3.1.2 Justification 28
3.2 Data Acquisition and Preprocessing 28

3.2.1 Source of Structural Engineering Documents 29
3.2.2 Types of Inputs and Preprocessing Steps 29
3.2.3 Integration with Downstream Systems 30
3.3 Tooling and Frameworks Used 33
3.3.1 Core Frameworks 33
3.3.2 Document Management Tools 33
3.3.3 Vector Store & Embedding 33
3.3.4 Interface and Logging 34
3.3.5 Limitations and Assumptions in Tool Selection 34
3.4 System Architecture and Component Design 34
3.4.1 Overview of RAG in Context 35
3.4.2 Modular Breakdown of the Framework 35
3.4.3 Role and Backstory of Each Agent 36
3.4.4 Task Design Philosophy 37
3.4.5 Integration of Agents via CrewAI 38
3.4.6 Flow Diagram of Sequential Processing 39
3.4.7 Scalability Considerations 41
3.4.8 User Interaction 41
3.5 Evaluation 42
3.5.1 Hyperparameters 43
3.5.2 Evaluation Strategy 44
4 Results 45
4.1 Robustness Analysis 46
4.2 Accuracy Analysis 48
4.3 Processing Time 50
4.4 Qualitative Behavior 51
5 Discussion 55
5.1 Study Limitations 57
5.2 Future Recommendations 58
6 Conclusions 60
References 62

Preface and acknowledgements

Completing this thesis has been a period of learning and personal growth. I am very grateful to my supervisor, Mohamed Noureldin, for his guidance, clear feedback, and the confidence he showed in me throughout this process. I would also like to thank my advisor, Petri Kortelainen, for his helpful suggestions. His support during important stages of the research improved both the quality and clarity of the work. Many thanks to Ramboll for funding this project and for giving me the chance to work on such an exciting topic.

A special thanks to my friends for their encouragement and assistance whether through discussions that clarified tough points or simply by keeping my spirits up when things got stressful.

I am grateful to my family for standing by my decision to study abroad and for their unwavering support throughout this journey. And I would like to express my forever gratitude to Allah for giving me the strength and perseverance to complete this research.

Otaniemi, 29 September 2025
Hafiz Muhammad Ahmad

Symbols and abbreviations

Abbreviations

AI	Artificial Intelligence
LLMs	Large Language Models
RAG	Retrieval Augmented Generation
GPT	Generative Pre-trained

1 Introduction

Structural design seeks to create safe, usable, and durable structures within clear professional rules. Every design step must follow the building code, and the resulting choices must be transparent and verifiable. In Europe, the Eurocodes set the common framework, while each country issues a National Annex that adjusts parameters to local conditions. In Finland, engineers must apply the Finnish National Annex when checking actions, factors, and detailing rules. Cross-checking these clauses is slow and prone to error. Small deviations in national parameters can undermine safety and regulatory acceptance.

Recent advances in artificial intelligence suggest a new form of assistance. Large language models can read technical text and answer questions in natural language. They generalize from examples and often respond with little task-specific training (Raiaan et al., 2024). The transformer architecture replaced recurrence with attention and enabled robust learning of relations within long texts. It scales well and supports strong generalization across tasks (Vaswani et al., 2017). This raises the possibility of AI assistants that help engineers interpret codes. However, general models lack reliable knowledge of specific national annex clauses. Their training data may include general Eurocode information, but they often miss the granular, country-specific details and thus might provide generic or incorrect advice. Also, they often fabricate details or copy patterns from unrelated sources. The literature notes that such hallucinations come from gaps in training data and from the absence of grounding in verified documents (D'Antonoli et al., 2024). These shortcomings become critical in code compliance contexts, where precision is paramount. An LLM might confidently answer a structural query with a plausible-sounding Eurocode rule, yet if it ignores the Finnish Annex's unique values, the result could violate local regulations. When the controlling provision comes from the Finnish annex, using the general Eurocode can lead to incorrect choices. This risk grows when answers are sought from generic web sources or general language models that do not know the local annex and present confident but wrong explanations. These limits conflict with the standards of traceability and accountability that guide engineering design and review.

Retrieval augmented generation offers a direct response to these limits. In a retrieval augmented system, the model conditions its output on passages retrieved from an external corpus. When the corpus contains the authoritative texts, the answer can be checked and traced. Reviews of retrieval augmented generation point to the key design choices. Corpus curation must focus on the sources of truth. Retrieval must be precise at the clause level. Prompts should instruct the model to cite the exact clause. Verification steps should compare the draft with the retrieved text and reject unsupported statements (Nguyen, Dang, & Pham, 2024). These ideas align with engineering practice, where the normative text and the national annex control the decision. Retrieval therefore provides a technical mechanism to anchor language model

answers in Finnish annex provisions, and to update behavior when the corpus changes (Kampelopoulos et al., 2025).

Multi-agent systems add structured reasoning around retrieval. Instead of a single pass from question to answer, agents can plan, search, verify, and revise. Studies show that specialized agents can coordinate to improve reliability on complex tasks and technical queries (Hughes et al., 2025).

Despite growing interest in AI for the structural engineering, the applications of RAG and multi-agent workflows remains underexplored. The literature highlights a gap for localized, clause-level assistance that is grounded in national provisions. This thesis addresses that gap by focusing on the Finnish National Annexes. It develops a multi-agent retrieval-augmented assistant that first retrieves clause-level evidence from the annex corpus and, only if needed, falls back to general Eurocode knowledge. The design aims to reduce hallucinations, improve traceability, and align with the documentation practices used in structural design reviews. By equipping the LLM with the ability to look up the annex text on the fly, we ensure its output remains anchored to the authoritative documentation.

1.1 Research Objectives and Questions

The research aims to bridge LLM technology and structural engineering practice. The key objectives are: (1) to develop a prototype AI assistant that can automatically retrieve and present country-specific code information in response to user queries, and (2) to evaluate its performance relative to single-agent or non-augmented LLM approaches. In line with these objectives, we address the following research questions:

Research Question 1: How can a multi-agent RAG system be designed to integrate National Annex knowledge into LLM responses, and does it effectively provide accurate answers that adhere to country-specific code clauses?

Research Question 2: To what extent does the inclusion of a retrieval agent improve the grounding and accuracy of answers compared to an LLM relying solely on its pretrained knowledge?

Research Question 3: What are the system's limitations in practical use and how do different LLM configurations impact its reliability and completeness?

The research questions in this study were developed to address a clear gap in current AI applications for structural design. Most language models today are not able to reliably answer queries that rely on localized or code-specific knowledge, such as those found in the Finnish National Annex. These models often produce generic or incorrect answers when faced with technical standards. By focusing on integrating structured information from national annexes, this research investigates whether a multi-agent RAG system can provide more accurate and traceable responses. The goal is to move from general-purpose reasoning toward domain-specific understanding. This shift may reveal gaps in current compliance practices and support better decision-

making by engineers who rely on precise documentation. Instead of depending only on pre-trained models, this approach allows for direct access to verified sources, improving both transparency and trust in the results.

1.2 Structure of the Thesis

This master thesis is structured into four main sections: a literature review, system implementation and methodology, evaluation of the multi-agent RAG system, and a discussion with conclusions and future recommendations.

Chapter 2 provides background on LLMs in engineering and related work on RAG and multi-agent systems, grounding our approach in the literature.

Chapter 3 details the methodology, including the system architecture, the preparation of the Finnish Annex knowledge base, and the experimental setup for evaluation.

Chapter 4 presents the results of our experiments, including quantitative metrics for answer accuracy, robustness and latency, as well as qualitative analysis of answer examples.

Chapter 5 offers a discussion of these results. We interpret the findings, compare them with expectations and prior studies, and delve into the limitations of the current system.

Chapter 6 outlines future work and practical recommendations for improving and extending the system. Finally, Chapter 7 concludes the thesis by summarizing the contributions and reflecting on the broader implications for integrating AI into structural design workflows.

2 Literature review

Structural engineering is increasingly influenced by advancements in artificial intelligence- While LLMs excel in natural language understanding, reasoning, and code generation, their direct application in engineering remains limited by issues of factual accuracy, domain specificity, and transparency. These shortcomings become critical in contexts requiring precise compliance with localized building regulations, such as the Finnish National Annexes to Eurocodes, where even minor deviations from prescribed values or clauses can compromise safety and compliance. Retrieval-Augmented Generation (RAG) offers a means to mitigate these limitations by grounding LLM outputs in verified external sources, enhancing both accuracy and traceability. When combined with autonomous AI agents, this approach enables dynamic query refinement, iterative reasoning, and integration with engineering design tools, which is collectively referred as Agentic RAG. Despite promising developments, the integration of these technologies into structural engineering practice remains underexplored, particularly in automating compliance checks for detailed, country-specific regulations. This literature review explores these LLMs, RAG, AI agents, and their applications, highlighting the need for innovative frameworks that integrate LLMs into structural engineering.

2.1 Large Language Models (LLMs) in Engineering Applications

The development of LLMs is an important direction in the progress of Artificial Intelligence. The pre-trained generative LLMs have shown remarkable proficiency in the use and analysis of natural language and have demonstrated an emergent ability of logical judgment and reasoning by offering crucial support for the development of Generative Artificial Intelligence applications. Large Language Models (LLMs) are capable of understanding and generating human-like text. They acquire language patterns and structures through extensive training on massive datasets gathered from different sources, including web content, books, articles, and code repositories. The capacity to deal with language makes them beneficial for several activities, including engineering. This section will cover the basics of LLMs, their capabilities, limitations, and applications in structural engineering.

2.1.1 Fundamentals of LLMs

LLMs are based on deep learning techniques that represent a significant advancement in artificial intelligence. These models are built on a transformer type neural network which was introduced by Vaswani in 2017. This neural network architecture excels at handling data sequences, such as text, and understanding the context of each word in respect to the other words in the sequence. The Transformer uses a self-attention mechanism which allows the model to weigh the importance of different words in a sentence while processing a specific word. This allows the architecture to develop a thorough

awareness of sentence structure and the nuanced meanings of words in their specific contexts which was difficult for older models like Recurrent Neural Networks (RNNs) (Vaswani et al., 2017). This is very important for understanding complex technical documents, like engineering reports or building codes.

LLMs undergo pre-training on the large datasets in self supervised manner. During pre-training, the model learns to predict missing words in a sentence or the next token in a sequence (Göpfert et al., 2024). This process enables the LLM to develop broad understanding of language, grammar, and world knowledge (Raiaan et al., 2024). For example, GPT-3 demonstrated that after wide range training, it could perform new tasks in a zero shot way without specific fine-tuning (Brown et al., 2020). One of LLMs' distinguish advantages is their ability to be fine-tuned for certain purposes. Fine-tuning is a process that supplements a pre-trained model with thousands of labelled examples specific to the target task. A general-purpose LLM can be trained on a smaller dataset to improve performance for a specialized task by optimizing its weight parameters. For instance, an LLM can be fine-tuned on structural engineering codes and literature to improve its ability to address structural analysis and design queries. Fine-tuning is what gives LLMs their versatility and power.

2.1.2 History of LLMs

The history of language modelling began in the 1950s when AI was established as an academic discipline and machine translation experiments. Some of the first language models in the 1960s, such as ELIZA, were rule-based systems which used pattern matching to imitate conversation. These earlier models were unable to handle the complexities of human language (D'Antonoli et al., 2024). Statistical models and n-gram models were also developed at this time to predict word sequence probability from their frequency. However, such models had difficulties with long-term dependencies and contextual information (Raiaan et al., 2024).

The development of neural networks resulted in more complex methods for language modeling. Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks that emerged in 1997 were crucial milestone in support of modelling sequential data like language and preserving context information more effectively (Raiaan et al., 2024). The trend shifted towards scaling up model parameters and training data significantly after the success of transformer-based models resulted in the creation of LLMs with billions to hundreds of billions of parameters. Models like GPT-3 exhibited impressive zero-shot transfer learning capability, in which they performed tasks without task-specific fine-tuning. This results in creation of numerous LLMs by enhancing architectures and training procedures (Naveed et al., 2023). There has been an increasing move towards open-source and instruction-tuned models. The development of LLMs originates from early natural language processing techniques and has advanced through multiple key stages.

The application of LLMs in design and automation is still in its early stages, but it is growing quickly. Initially, LLMs were tried out for simple tasks such as generating text and answering questions. As they have become more powerful, the extent has expanded to perform complex tasks like writing code, reasoning, generating new ideas and tool usage in a variety of domains. In the field of engineering, LLMs are starting to play a role in automating geometry creation by converting textual inputs into design changes, especially for parametric designs. Therefore, they can be used to automate design processes, develop concepts, ideate design, which will save time and reduce human errors ultimately.

LLMs' ability to analyze and understand complex engineering documentation, such as codes and standards, is considered as a significant opportunity to automate repetitive and administrative tasks, allowing engineers to focus on more creative problem solutions. The progress continues with the emergence of multimodal LLMs with the ability to process textual and image inputs, offering AI with "vision" to detect and rectify errors in geometric outputs. The most fascinating development of this sort is the development of AI agents using LLMs. These agents are AI systems that are not only able to understand and generate texts but can also execute actions in the real world. For example, An AI agent could be used to search information regarding particular building materials, analyze their properties and then use that information to create designs for a new structure or make a comparison. This is a significant step towards a future when AI can collaborate with human engineers to solve complex problems.

The success of LLMs is a result of advancements in deep learning, compute power, and availability of huge volumes of training data (Naveed et al., 2023). LLMs are now a key element of modern NLP pipelines, which has the ability to perform better even with smaller quantities of labelled data.

2.1.3 Capabilities and Limitations for Engineering Tasks

Because they are trained on so much text, LLMs accumulate a wide range of knowledge including some basic engineering and scientific facts found in literature. They excel at natural language understanding and generation, meaning they can parse complex technical descriptions and produce well-formed responses. For engineering, this is valuable in several ways. First, LLMs can serve as intelligent assistants that answer questions or explain concepts. An engineer could ask an LLM to summarize a clause from a building code or to explain the difference between two structural analysis methods. The model, if it has seen similar text, can provide a helpful summary or explanation. LLMs have also demonstrated an ability to do elementary reasoning and step-by-step problem solving using the knowledge encoded in their training data. For example, they can be prompted to perform unit conversions, solve simple structural analysis problems, or check calculations, although their reliability in detailed engineering math is still imperfect. One emerging capability important to engineering is code generation. Modern LLMs like GPT-4

have been used to write programming code given a description of the task. This means an engineer could describe a simulation setup or a design calculation in plain language, and the LLM could produce code (e.g. Python, MATLAB) to perform that task. Early studies have shown promise in this area – for instance, researchers have used ChatGPT to generate finite element analysis code for simple problems (Qin et al., 2024). This suggests LLMs could eventually automate parts of the engineering workflow that involve coding or repetitive calculations.

Another intriguing use of LLMs is in geometry and design automation. Because they handle language, LLMs can take instructions like "increase the beam size by 20% in the model" and potentially interface with CAD or BIM software to make those changes. In preliminary experiments, LLM-driven agents have been used to convert textual descriptions into modifications of parametric design models (Qin et al., 2024). This hints at a future where an engineer might simply tell the computer the design changes needed, and the AI executes those changes in the digital model. Generative design is another area of interest: given high-level goals (e.g. minimize weight while respecting code limits), AI could generate design options for the engineer to consider. We are already seeing research that combines LLMs with other generative AI techniques (like GANs or graph neural networks) to create structural design alternatives rapidly (Liao et al., 2024). For example, generative models have been used to propose preliminary layouts of shear wall structures in buildings, which can then be optimized by the engineer or another algorithm (Liao et al., 2024; Qin et al., 2024). LLMs complement these techniques by interpreting the engineer's intent and controlling the process.

Despite their impressive capabilities, LLMs have notable limitations that must be understood, especially in high-stakes fields like structural engineering. One major issue is accuracy. LLMs do not truly understand the facts they present; they generate responses that sound plausible based on patterns in data. This can lead to hallucinations, where the model confidently states incorrect information. In an engineering context, a hallucinated answer (e.g. a nonexistent code clause or a wrong formula) could be dangerous. The models also have a static knowledge cutoff, they only know information up to when their training data was collected. They won't inherently know about newest standards, recent projects, or emerging research unless specifically updated. Another limitation is that LLMs lack explicit domain knowledge unless it was in their training data. General models might not include specialized content like a specific country's building code or niche structural engineering guidelines. For instance, a general LLM might know broadly about "Eurocode 2" (concrete design) from public sources, but it might not know the exact provisions of the Finnish National Annex to Eurocode 2 if that text wasn't part of its training. This means out-of-the-box LLMs could miss critical local adjustments like specific safety factors or climate considerations in Finland. In practice, this has been observed that a general model will sometimes give correct generic advice, but omit that, say, in Finland the recommended value for some load factor is different. Such gaps are unacceptable in structural

design, where compliance with every applicable clause is mandatory. Addressing this requires feeding the model with the relevant documents or using techniques we discuss later like retrieval augmentation to compensate for what the model doesn't know.

2.1.4 Existing research of LLMs in structural engineering

Despite the limitations, there is a growing body of research on the use of LLMs in structural engineering. LLMs are being explored for the design and optimization of shear wall structures, where it may serve as a controller, interfacing with engineers in the form of language, translating descriptions into code, and invoking methods for structural generation and optimization. This could include factors like empirical rules, mechanical performance, and material use. LLMs-based solutions can be used throughout the process, from processing architectural drawings to generating structural plans and creating analytical models. Research shows that there is a significant increase in speed when compared to traditional approaches. Generative AI such as techniques like GANs and GNNs is applied in the design stage first, and optimization according to user specifications with respect to parameters such as material usage and mechanical parameters follows (Qin et al., 2024). Evidence exists of research into employing generative AI based on deep reinforcement learning for generation and optimization of steel truss structure topologies. Intelligent design has seen results meeting code requirements and achieving material use similar to engineer designs, while there still exists a vast gap in systematic research into the development and deployment of generative AI, in particular, in building structural design, which leads to discontinuous results and challenges for researchers (Liao et al., 2024).

LLMs have also the ability to transform structural analysis systems by automating tedious repetitive processes and compliance checks against building code requirements. They can assist engineers in making informed decisions about materials and design options (Pokhrel & Parajuli, 2023). It has been investigated whether LLMs can be combined with methods such as the Finite Element Method (FEM) to iteratively improve truss constructions (Jadhav & Farimani). In civil engineering, there is a focus on AI applications for material and structural assessments. Research is developing frameworks that use prompt engineering to integrate user domain knowledge about vibration signals with LLMs' inference ability for structural health monitoring. This involves processing vibration data to identify the actual states of structures. LLMs can offer insights into the dynamic characteristics of vibration signals that may indicate structural states. They can also help with programming tasks like creating feature extractors for vibration signals or building machine learning classifiers to recognize structural states. This method aims to develop practical SHM applications without needing advanced programming or math skills (Nguyen, Dang, & Pham, 2024).

In construction safety, LLMs are being used to improve workplace safety. Some applications include serving as training tools in virtual reality, spotting

safety hazards in images, and reviewing accident reports. LLM performance on standard safety certification tests shows strengths in safety management systems and hazard identification, but weaknesses in science, mathematics, emergency response, and fire prevention (Sammour et al., 2024). Although the research on leveraging LLMs to analyze construction planning constraints from text is still ongoing. The main research questions in this area include finding constraint discussions automatically, examining stakeholder discussion patterns, and collecting important information for decision-making (He et al., 2024). Additionally, in the broader AEC industry, LLMs have been used in deep learning frameworks to automate and improve the accuracy of earthwork quantity estimation. While such LLM-based approaches can show superior performance, they require significantly more computational resources (Kang & Kang, 2024).

There is a significant gap in the current use of LLMs in structural engineering. This gap is especially noticeable when it comes to integrating specialized, country-specific standards and annexes. General-purpose LLMs are effective but often miss the detailed knowledge needed for national building codes, like the Finnish national annexes to Eurocodes. This limitation can lead to wrong or incomplete responses in localized regulatory areas, which is unacceptable in a field where accuracy and compliance are crucial. This thesis aims to fill this gap by developing an AI-powered tool called the Structural Engineering Assistant. This tool is designed to give specialized support for structural engineers by incorporating that localized knowledge. The following sections will delve into approaches to address these challenges, such as combining LLMs with retrieval systems and incorporating them into autonomous agents.

2.2 AI Agents and Retrieval-Augmented Generation (RAG) in Design Workflows

Beyond the basic abilities of LLMs to understand and generate text, the field of artificial intelligence is moving towards more autonomous systems called AI agents. These agents can process information, plan, act, and interact with their environment to reach specific goals. Moreover, Retrieval Augmented Generation (RAG) is an important technology that enhances the capabilities of these agents or LLMs, particularly in fields that require current, precise and dynamic information, such as structural engineering.

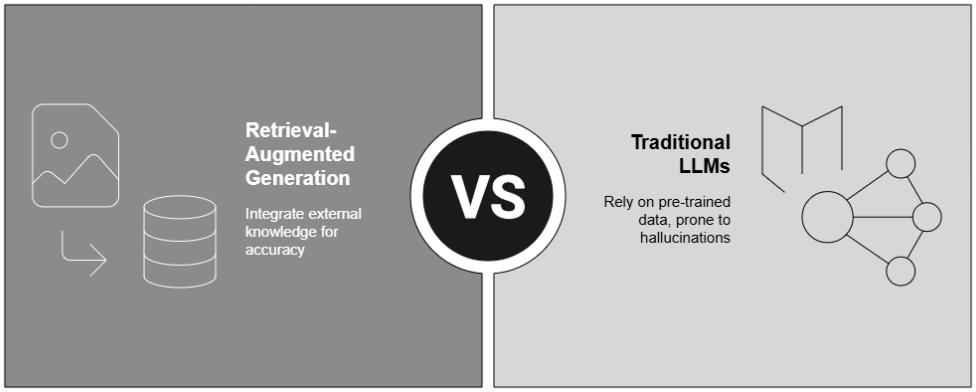

Figure 1 Comparison between RAG and Traditional LLMs

This section will introduce AI agents, explain the concept of RAG, and discuss their combined role in modern design workflows.

2.2.1 Theoretical foundations of Retrieval-Augmented Generation (RAG)

Despite having remarkable abilities, LLMs often struggle with hallucinations, and their knowledge is limited because of their static training data. This makes it hard for them to answer real-time questions. Retrieval Augmented Generation (RAG) emerged as a key solution.

The foundation of RAG is its hybrid approach. It combines information retrieval with language generation to enhance the capabilities of LLMs. This approach was created as a response to some of the key limitations of models relying solely on their pre-trained static data, such as producing plausible but inaccurate information (hallucinations), having outdated knowledge, and lacking domain-specific expertise. The main idea is that with dynamically access to relevant external knowledge, the generation process can be more factual, real-time, or domain-specific data (Gupta, Ranjan, & Singh, 2024).

The integration of retrieval and generation in RAG typically involves a pipeline with two main components: a retriever and a generator (Zhao et al., 2024). The process also includes an augmentation step that bridges these two components and is considered as the interaction between the retrieved results and the generator (Singh et al., 2025).

1. **Retrieval Phase:** When a user inputs a query, the retriever component searches for relevant information from an external knowledge source. This source can be a database, vector store, or collection of documents (Zhao et al., 2024). Then, it usually extracts information by measuring the similarity between the input query and text chunks or documents using vector embeddings (Prabhune & Berndt, 2024). Advanced retrievers utilize dense vector search and transformer-based models to improve precision (Singh et al., 2025).

2. **Augmentation Phase:** The information that is retrieved is then used to augment the input to the generator (Zhao et al., 2024). This is the most critical step where the retrieved content is merged with the original query to form an integrated input in order to provide additional context to LLM (Prabhune & Berndt, 2024).

3. **Generation Phase:** Lastly, the generator takes the augmented input and then produces the answer (Zhao et al., 2024). The LLM integrates the retrieved documents into contextually suitable and coherent response. Ideally, it will quote or closely follow the retrieved information, significantly improving accuracy and trustworthiness.

Figure 2 Workflow of the Retrieval-Augmented Generation (RAG) Process

The major purpose of this integration is to maximize the strengths of retrieval and generation while minimizing their shortcomings. Retrieval systems excel at identifying relevant material but have traditionally struggled to synthesize new content. On the other hand, Generative models (LLMs) are fluent and creative but can "hallucinate" or provide outdated information because their knowledge is limited to their training data. RAG addresses this by emphasizing external knowledge in LLM replies. It accesses only relevant, real-time information from external sources. Additionally, advances in RAG introduce features like dynamic retrieval strategies, iterative reasoning, tool applications, and multi-agent interaction. These improvements assist in selecting and using external knowledge for generation (Prabhune & Berndt, 2024).

The concept of RAG was formalized by Lewis et al. (2020) in a seminal paper (Gao et al., 2023), but it builds on a simple idea that predates LLMs using

search engines or databases along with AI to get facts. What's new is how tightly integrated it can be. The retrieval can be optimized and even learned jointly with the generation model in some implementations. There are advanced RAG techniques now that go beyond the basic pipeline. For example, some systems perform iterative retrieval, where the LLM might say, "I need more info on X" after seeing initial results, and then retrieve again. Others incorporate tool use if a retrieved document has a formula, the LLM agent might plug numbers into a calculator tool to get a result, then continue generating the answer. In summary, RAG provides a flexible framework that combines the parametric knowledge inside an LLM (what it "knows" from training) with non-parametric knowledge (external sources). Gao et al. (2023) note that this hybrid approach is key to deploying LLMs in specialized domains where the model handles language and reasoning, while the retrieval ensures factual correctness and domain coverage (Gao et al., 2023).

2.2.2 Introduction to AI Agents and Agentic AI

An AI agent is essentially an autonomous entity that observes its surroundings and takes actions to achieve specific goals (Hughes et al., 2025). In the past, these agents were created for limited tasks, had little independence, and depended on set rules or logical reasoning. Consider them as systems designed to carry out particular tasks, that often need human help for setup or complicated steps. They can use LLMs and other AI technologies to complete these tasks on their own, but usually within a specific scope. For example, a simple AI agent may sort emails or assist customers by responding to predefined inquiries (Sapkota, Roumeliotis, & Karkee, 2025).

Agentic AI is a more advanced approach as compared to AI agents. This is the vision of AI that self directs itself. It builds on generative AI and AI agents by providing more advanced reasoning and interaction capabilities. These systems can work independently for achieving larger goals and can also complete complex tasks that involve planning and reflection (Schneider, 2025). Unlike basic AI agents, which only respond to prompts, agentic AI exhibits proactivity, flexibility, and goal-oriented behavior. It continuously adjusts its operations to achieve set goals with minimal human interaction. This represents a shift from AI being just an assistive tool to becoming an active collaborator (Murugesan, 2025). The concept of autonomy in agentic AI goes beyond just completing one task. It involves breaking down larger objectives into smaller goals and adjusting strategies to reach them. These systems act as active partners. They have reduced human interactions by improving their expertise and decision-making instead of just providing assistance (Acharya, Kuppan, & Divya, 2025).

Table 1 Comparison of Generative AI, AI Agents, and Agentic AI

Aspect	Generative AI	AI Agent	Agentic AI
Reasoning	Primarily immediate responses	Rule based or supervised	Iterative planning and reflection

Interaction	Mostly only with the user	User, tools (external APIs)	User, tools, real-world, other AI agents
Execution	Single-step tasks, content generation	Task-specific execution using tools	Workflows, sequence of actions, goal achievement
Autonomy	User-driven, limited	Task-specific, constrained	Self-directed, dynamic, often multi-agent
Adaptability	No self-improvement, bound to training data	Learn within their specific domain	Collects and leverages experiences, adapts across tasks and environments
Knowledge Base	Internal, static pre-trained data	Can access external data via APIs	Integrates diverse data sources, persistent memory

The convergence of RAG and Agentic AI has led to the creation of Agentic RAG. This model integrates autonomous agents directly into the RAG pipeline, changing static retrieval workflows into dynamic and adaptable ones. In an Agentic RAG system, LLM agents manage the retrieval process. These agents understand user queries well, plan retrieval strategies on the fly, and filter and refine retrieved information iteratively (Singh et al., 2025). They use various tools based on their needs such as vector search engines, web searches, calculators, and APIs to gather information. They have also ability to adjust their reasoning to improve information retrieval and synthesis, often through multiple rounds of reasoning and retrieval cycles. They verify the final output's context and accuracy. This close connection of reasoning and tool use lets Agentic RAG handle complex, multi-step tasks that traditional RAG or standalone LLMs cannot manage (Cheng et al., 2025).

Agentic AI often involves Multi-Agent Systems (MAS), where several AI agents cooperate to accomplish common objectives. This system operates by dividing the main goal into smaller, manageable tasks. These tasks are assigned to different agents according to their abilities and skills. This approach boosts the system's efficiency, scalability, and fault tolerance. The agents follow structured protocols to coordinate their actions and communicate well. They can also share context and improve responses through ongoing feedback. This targeted, task-focused intelligence further increases the system's efficiency, scalability, and fault tolerance. Each agent concentrates on its specific role, leading to clearer reasoning and better understanding. An example of such a system is a corrective RAG workflow (Ravuru, Sakhinana, & Runkana, 2024). This workflow involves a context retrieval agent, a reranking agent, and a corrective agent that work together to find and fix misinformation (Singh et al., 2025).

2.2.3 Role of RAG and Agentic RAG in Improving AI Decision-Making

The main advantage of RAG in decision-making is its ability to improve the accuracy of answers and reduce the chances of hallucinations by extracting the responses from provided and verifiable sources (Gupta, Ranjan, & Singh, 2024; Prabhune & Berndt, 2024; Singh et al., 2025; Zhao et al., 2024). This grounding in factual data is crucial for critical applications that demand high accuracy, such as code compliance design or queries (Hughes et al., 2025). RAG also helps in updating knowledge and there is no need to retrain the entire LLM which makes it more practical for fields where information changes quickly (Prabhune & Berndt, 2024).

While traditional RAG improves LLM by connecting outside sources, it still faces limitations. They usually have fixed workflow and may not be able to refine their searches depending on intermediate results or user comments, which can lead to incomplete and disconnected responses. Agentic RAG addresses this issue by taking the main idea of RAG and integrating it with the decision-making adaptive capabilities of autonomous agents. They can dynamically modify retrieval strategies based on the complexity of query or the relevant information retrieved. They can also participate in iterative refinement loops, in which they self-reflect, analyze their own results, and update queries in order to obtain better evidence. This continual feedback loop increases the accuracy and relevance of responses (Singh et al., 2025). For example, one agent could be in charge of retrieving data, another of optimizing it, and a third of verifying the final result.

RAG has a wide range of applications throughout various fields. In construction management, RAG4CM system helps to retrieve accurate information from several documents. This addresses problems of granularity mismatch and ambiguities (Wu et al., 2025). Agentic RAG has great potential to change how structural design workflows operate. It can help engineers ensure their designs meet various building codes. Instead of manually going through hundreds of pages, the agent can quickly extract relevant sections of the code based on the query and design parameters. It can then use the capabilities of LLM to interpret and apply those regulations to specific design challenges. Another use could be in material selection and optimization. One agent can specify design constraints, while other agent can use RAG to find appropriate material properties and cost details from external databases, then generate a response. This can significantly speed up the early design process and lead to better and more sustainable material choices (Dyke et al., 2024). They can also play an important role in iterative design. When design changes happen, the agent can continuously retrieve and evaluate new information, offering real-time feedback to make solutions more robust.

RAG provides the knowledge backbone for trustworthy AI outputs, and agentic AI provides the problem-solving and autonomy. Together, they form an "AI coworker" that can handle sophisticated tasks: finding not just any

answer, but the correct answer with justification, and doing so through a chain of logical steps and tool uses that mimic an engineer's approach. For this thesis, these concepts are foundational which aims to build a Structural Engineering Assistant. We plan to use retrieval augmentation to give the assistant access to Finnish building codes texts that general models lack and incorporate agent-like behavior so it can plan out multi-step checks or design recommendations. The ultimate goal is to improve decision-making support by AI that not only answers questions but actively contributes to design workflows, safely and transparently.

2.3 AI in Structural Engineering

Artificial Intelligence techniques have been gradually transforming aspects of structural engineering over the past decade. Initially, applications were focused on specialized sub-domains like structural health monitoring and material science, but with the improvements in AI algorithms, better computing power, and the large amount of data available, we're seeing broader adoption. This section reviews some notable current trends, and then discusses specific challenges, especially those relevant to using generative AI for structural design and finally touches on efforts to use AI for code compliance checking.

2.3.1 Current Trends in AI for Structural Engineering

Structural health monitoring (SHM) is one of the key areas where AI is used for aging infrastructure, such as bridges and buildings. This technology allows for better monitoring to ensure safety and extend lifespan. Traditional SHM methods usually rely on manual inspections and basic sensors, which have limitations for large and complex structures (Salehi & Burgueño, 2018). AI helps us move beyond these challenges by using machine learning models that can analyze data to classify it, detect anomalies or damages, and even predict responses. Early work used statistical pattern recognition on features extracted from sensor signals (Thai, 2022). Now, with deep learning, SHM has embraced models like Convolutional Neural Networks (CNNs) for vision-based monitoring. A clear example is using drones or cameras to take images of a bridge and then a CNN automatically identifying cracks, spalling, or other surface defects. Cha et al. (2017) demonstrated a CNN that could detect concrete cracks from images with high accuracy, eliminating the need for manual feature extraction. Vision-based SHM means we can continuously or frequently inspect large structures via images or video and let AI highlight the areas of concern, which is far more scalable than having engineers manually review thousands of photos (Cha, Choi, & Büyüköztürk, 2017).

Similarly, AI is an effective tool for predicting the mechanical properties of construction materials, such as the compressive strength of various concrete types. In conventional methods, it requires lab tests that take weeks to cure cylinder and break. Machine learning offers a way to predict material properties based on mixed designs, curing time or early age measurements. This helps in optimizing concrete mix design and predicting material

performance, hence making construction more efficient (Thai, 2022). AI also plays an essential role in analyzing how structures behave during earthquakes. Engineers use AI to predict how buildings and bridges will respond to seismic forces, which aids in the design of earthquake-resistant structures and detection of seismic damage (Sargiotis, 2024).

2.3.2 Challenges in Generative AI for Structural Design

Gen AI has great potential to make structural engineering safer, more efficient, and more sustainable. LLMs are key players in these systems. They can learn from existing designs, create new ones and understand the language of engineers, translate it into code, and automate design tasks. They can even optimize designs based on empirical rules, mechanical performance, and material use. Despite these advantages, there are important challenges. These include the need for high computational resources and concerns about accuracy and transparency (Spencer Jr et al., 2025).

Issues related to data are significant. Gen AI models require large amounts of high-quality data to learn (Sargiotis, 2024). This means they require access to many past designs, performance data, and failure analysis. Such data is difficult to obtain and may not always be accurate. If the training data contains biases, the AI's designs might also be biased, resulting in unfair or unsafe outcomes (Abduljaleel et al., 2025; Kampelopoulos et al., 2025). Data privacy and security are also crucial. AI systems sometimes may require access to sensitive project data. We must ensure this information is safe from misuse or cyber threats. It's often unclear who is liable if an AI system makes a mistake that causes an accident or structural failure. Legal frameworks need to be updated to clearly define accountability for AI-related incidents (Ghimire, Kim, & Acharya, 2023; Sargiotis, 2024).

LLMs, as deep learning models, operate largely as “black boxes”, that make them hard to understand how they came up with solution. They do not provide explanations for why they gave a certain answer unless they are prompted to produce a rationale, which itself may or may not be trustworthy. This lack of transparency is at odds with engineering practice, where decisions need to be justified by calculations or code references. For structural engineers, explaining and justifying every design choice is essential. An engineer will not want to trust a design recommendation from an AI unless the reasoning can be checked. Therefore, a key theme in applying LLMs to engineering is finding ways to ground their outputs in verifiable sources and to extract explanations, so that the human expert remains in control of the validation. Explainable AI (XAI) is an active research area that aims to make these decisions clearer (Liang et al., 2024; Sargiotis, 2024). Lastly, integrating AI into an existing workflow is challenging since it may require new skills to effectively apply and validate Gen AI outputs. Future structural engineers will need to be proficient not only in tradition engineering principles but also in machine learning and AI ethics For LLMs, the quality and accuracy of the generated output can be very sensitive to the precise wording of the input

prompt. Even small changes in phrasing can significantly change the results. Sometimes, this can result in nonsensical or fabricated facts if the AI doesn't have a strong grasp of the structural domain. This makes creating effective prompts a tough job for engineers without specialized AI knowledge (Ghimire, Kim, & Acharya, 2023; Nguyen, Dang, & Pham, 2024).

2.4 Training AI for Structural Design Code Compliance

Structural engineering practices revolve around design and review cycles. These cycles involve a series of steps where structural engineers develop, assess and refine structural design solutions. This process is rarely straightforward and often includes rounds of design, checking, and revision. Ensuring code compliance is essential, as engineers must verify structural design and analysis against various code sections depending on the component of structure. This section will discuss the challenges of code compliance, how AI can be trained to understand and apply these codes, and the crucial role of explainability and interpretability in this process.

2.4.1 Overview of Structural Design Codes (Eurocodes and National Finnish Annexes)

Structural design codes are essential for ensuring the safety, reliability, and durability of buildings and other structures. In Europe, the Eurocodes are a set of harmonized technical rules for the design of buildings and civil engineering works. They cover various aspects, from the basis of structural design (EN 1990) to specific material codes like concrete (EN 1992), steel (EN 1993), and timber (EN 1995). The idea behind Eurocodes was to create a common set of standards across Europe, making it easier for engineers to work internationally and for products to be traded. However, the Eurocodes are not entirely uniform. Each member country can publish a National Annex, which contains country-specific parameters, values, and methods. These National Annexes allow countries to adapt the Eurocodes to their local conditions, such as climate, geological features, and established safety levels. For example, the National Finnish Annexes specify how certain aspects of the Eurocodes should be applied in Finland, reflecting Finnish building practices and environmental conditions. This means that while the core principles are shared, the exact application can vary significantly from one country to another.

One of the biggest slowdowns in the traditional design-review process, especially comes from incorporating National Annexes. While Eurocodes provide a common framework, each country has its own National Annex that modifies or adds to the main code. These annexes reflect local conditions, traditions, and specific safety levels. For engineers working on international projects or in countries with specific annexes, this means they can't just apply the general Eurocode rules. They have to carefully check the relevant National Annex for every design decision. This adds a layer of complexity and a significant bottleneck. It requires engineers to have detailed knowledge of

multiple sets of rules, which can be hard to maintain and apply consistently. Manual checking of these annexes is time-consuming and increases the risk of errors, as a small detail in an annex can have a large impact on a design. This is a clear area where current tools fall short, as they often don't seamlessly integrate these country-specific variations into the design and review workflow.

Training Large Language Models (LLMs) to understand and apply structural design codes, especially National Annexes, presents unique challenges. LLMs are powerful because they learn from vast amounts of text data. However, the data available for specific National Annexes is often limited compared to the general text data LLMs are usually trained on. This scarcity of data makes it difficult for LLMs to develop a deep and nuanced understanding of these specialized documents. Furthermore, National Annexes often contain highly technical language, specific numerical values, and complex cross-references that are not easily captured by general-purpose LLM training. The subtle differences and specific interpretations within these annexes can be easily missed or misinterpreted by an LLM that lacks sufficient, targeted training data. For instance, an LLM might understand the general principles of a Eurocode, but struggle with a specific partial factor or design method prescribed only in the Finnish National Annex. This is a critical limitation because even small errors in code application can have serious consequences in structural engineering.

Building codes are detailed legal documents that specify minimum requirements for the design and construction of buildings and structures. They cover a vast range of topics, including structural loads, material properties, fire safety, accessibility, and environmental considerations. The complexity arises from several factors: the sheer volume of information, the interconnectedness of different clauses, and the need for engineering judgment in applying general rules to specific design scenarios. Keeping up with code updates and variations across different jurisdictions is a continuous challenge for structural engineers. Manual code checking is a time-consuming and error-prone process. Engineers must meticulously review designs against hundreds or even thousands of code provisions.

2.4.2 Training LLMs to Understand and Apply Codes

Training current LLMs to understand and apply building codes involves several approaches. One method is to fine tune model for country specific standards. It simplifies the process of scaling the knowledge accessible to the LLM, as it avoids the need for complex retraining procedures. Fine-tuning, while useful in some scenarios, comes with its risks particularly the possibility of overfitting (Gao et al., 2023), which can reduce the model's ability to generalize beyond the specific dataset it was fine-tuned on. Furthermore, adjusting the parameters of a fine-tuned LLM can lead to unintended consequences that may affect the model's overall performance (Zhao et al., 2024). This approach also requires high computational resources. For many enterprise

applications, RAG presents a more sustainable and efficient alternative to fine-tuning, offering a scalable and reliable solution. This allows the AI to retrieve relevant code provisions for a given design scenario and even generate explanations for why a particular design is compliant or non-compliant.

2.4.3 The Role of Explainability and Interpretability

For AI to be effectively used in structural design code compliance, explainability and interpretability are crucial. Explainability refers to the ability of an AI system to provide clear and understandable reasons for its decisions. Interpretability, on the other hand, refers to the degree to which a human can understand the cause and effect of an AI system's internal workings. In a field like structural engineering, where safety is paramount, engineers need to trust the AI's recommendations and understand the basis for its compliance checks. If an AI system simply states that a design is compliant without explaining why, engineers may be hesitant to accept its output. An explainable AI system, however, could highlight the specific code clauses it checked, the parameters it used, and the calculations it performed to reach its conclusion. This transparency allows human engineers to verify the AI's reasoning, identify any potential errors or biases, and ultimately take responsibility for the final design. Without explainability, AI in code compliance risks becoming a 'black box' that undermines trust and accountability, which is unacceptable in a safety-critical domain (Liang et al., 2024).

3 Research material and methods

This section discusses the methodology applied in developing and testing the Structural Engineering Assistant, an AI-powered add-in to provide domain-specific support to structural engineers working with National Building Code of Finland. One of the key motivations for this effort stems from noticing the phenomenon that current large language models (LLMs), as proficient as they are, are often deficient in specific knowledge on national annexes to Eurocodes, e.g., Finland. To fill this gap, we developed a Retrieval Augmented Generation (RAG) system, enhanced by a multi-agent architecture, to give accurate and contextually relevant information.

3.1 Introduction to Methodological Framework

This methodological framework focuses on filling the knowledge gap of general purpose LLMs in specialized domains of structural engineering, particularly concerning to country specific standards. The primary objective was to create an intelligent assistant that can provide precise answers even when the knowledge required is highly specialized and not inherently present in the pre-trained knowledge base of the LLM. This approach justifies the use of RAG and multi-agentic system since it facilitates dynamic information retrieval and expert-level reasoning within a specified workflow.

3.1.1 Research Methodology Overview

This methodology involves four key steps which were followed in this study. These are:

1. To address the limitations of general purpose LLMs like hallucinations, outdated knowledge, and lack of domain specific information which in this case is unavailability of Finnish national annex by developing an organized knowledge base tailored to these purposes.
2. To develop a robust RAG system that integrates external, specialized information in LLM responses in a correct and contextually relevant sense.
3. To develop a multi-agent architecture with CrewAI that would allow a layered approach of information retrieval and reasoning to make the response more accurate and trustworthy by distributing tasks according to agent expertise (Zahedifar et al., 2025).
4. To establish a comprehensive and systematic evaluation strategies to compare the performance of different RAG implementations and the system in terms of quantitative and qualitative metrics (Zahedifar et al., 2025).

3.1.2 Justification

Traditional LLMs are proficient in general knowledge but they usually suffer from issues with very technical, constantly evolving, or proprietary information such as structural engineering's reliance on specific codes, where relying only on pre-trained knowledge tends to generate hallucinated responses unacceptable in precision-critical environments. RAG directly addresses these limitations by empowering LLMs to access and incorporate the pertinent, authoritative external data, relying on factual knowledge to significantly reduce errors, enhance credibility, and counteract the inherent hallucination-prone inclination while bridging gaps in existing or domain-specific knowledge (Fan et al., 2024). The scalability and flexibility of vector databases enable efficient storage and retrieval of large volumes of unstructured and semi-structured engineering documents through semantic search functionality that quickly discovers associated context even as knowledge bases expand, while RAG with vector search provides high-quality answers with explicit reference to source documents, which enables transparency, trust, and compliance in engineering decision-making (Siddharth & Luo, 2024). Furthermore, multi-agent architectures can empower sophisticated handling of complex queries through layered reasoning and error correction mechanisms, where deep-specialized agents specifically committed to retrieval, reasoning, and verification can cooperate outcomes and implement self-correction mechanisms, enabling nuanced problem-solving and knowledge retrieval that has proven to be extremely effective in advanced control engineering and other fields of technology (Zahedifar et al., 2025).

The system works based on a systematic-workflow process from document ingestion, where structural engineering documents like standards, guidelines, and national annexes are gathered, preprocessed, and split into logically coherent chunks enriched with metadata for traceability, followed by embedding and indexing processes that convert document chunks into vector embeddings using advanced embedding models and store them along with metadata in a vector database for semantic, scalable retrieval. User query handling accepts natural language queries through an interface, with the retrieval agent searching the vector database for the most relevant document chunks using semantic similarity, after which layered reasoning and response generation occurs as retrieved context is passed to the LLM to synthesize responses grounded in sourced material, while additional agents may validate responses, check compliance with standards, or provide fallback reasoning if retrieval fails. The system produces reference-backed output by showing users end answers with explicit references to source documents like file names, sections, and paths for transparency and auditability (Siddharth & Luo, 2024).

3.2 Data Acquisition and Preprocessing

The acquisition and preprocessing data pipeline are designed to automate, standardize, and optimize the ingestion of structural engineering knowledge

to facilitate AI-driven retrieval and reasoning. The RAG effectiveness greatly relies on having a well curated and processed knowledge base. Our system is designed to ingest and prepare structural engineering documents to facilitate optimal retrieval performance.

3.2.1 Source of Structural Engineering Documents

The primary source of documents in our knowledge base is the Finnish National Annexes. These documents are significant to provide accurate, code compliant information for structural design methods in Finland. The knowledge base may also include necessary technical documentation and guidelines.

3.2.2 Types of Inputs and Preprocessing Steps

Our system is designed to support a variety of document formats widely used in engineering contexts. This system supports .pdf, .md, and other document formats that are commonly used for technical standards and engineering documentation.

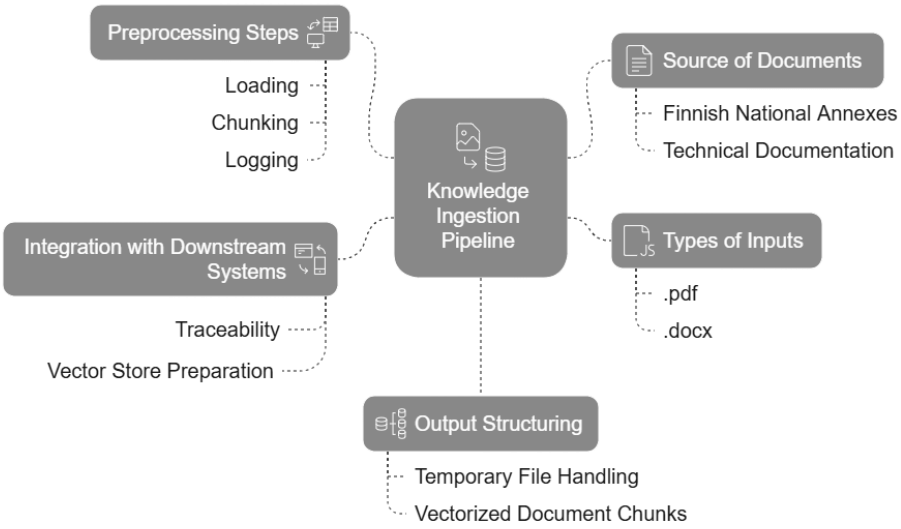

Figure 3 Structural Engineering Knowledge Ingestion Pipeline

The preprocessing pipeline involves several key steps:

Loading: The pipeline automatically traverses the entire directory, locating and loading all relevant files independent of nesting level. The documents are loaded with the suitable loaders (e.g., PyMuPDFLoader for PDFs, TextLoader for text files). Loaded documents are subsequently marked with required metadata, including a source_id unique ID, doc_type (determined from its directory layout), source_path, and filename. This metadata is

highly crucial for traceability as well as to provide references in the final responses.

Clause-Based Segmentation: To preserve the contextual structure of standards documents, we leveraged the documents' inherent clause numbering system. Each document was split into sections according to clause identifiers. By splitting at clause boundaries, we ensure that each segment corresponds to a specific clause or subsection, which is meaningful on its own. After splitting, any remaining text without an explicit clause heading is grouped under the preceding clause or a general section. Each text segment is associated with metadata: the source filename, the clause number (or "General" if none), and the page number. This meticulous preservation of metadata later allows the system to cite the exact clause and page in its answers, lending credibility and traceability to the responses.

Chunking: Following clause-based splitting, we further divided the text into smaller chunks to accommodate the token limitations of the embedding model and LLM. We used LangChain's RecursiveCharacterTextSplitter to chunk the text, with a chunk size of about 512 characters and an overlap of 128 characters between chunks. Chunk overlap ensures continuity, so that important information cut off at the end of one chunk is likely to appear at the start of the next chunk, preserving context. This splitter uses a hierarchy of separators (newlines, periods, punctuation, spaces) to ensure that chunks maintain contextual integrity, which is vital for effective retrieval (Qu, Tu, & Bao, 2024). This strategy is crucial for handling long documents: large texts exceed the context window of most models and need to be broken down so the model can effectively process and retrieve relevant sections. By splitting documents into semantically coherent chunks, we enhance the model's ability to find the most pertinent sections without losing key information that might span chunk boundaries. Each resulting chunk inherits the metadata of its parent document (filename, page, clause), so no matter how we split the content, we never lose track of its origin.

Logging: Strong logging systems are implemented throughout to monitor system operation, track agent interactions, and capture faults. It helps in debugging and analyzing the performance and make the system reliable. Error handling is designed to handle instances when no documents are found, APIs are not responding, or any other unforeseen issues with informative feedback to the user and the system administrators. This process is continuous for the whole to ensure that individual failures do not halt the entire ingestion pipeline.

3.2.3 Integration with Downstream Systems

Vector Store Preparation: After preprocessing, the next step is to convert each text chunk into a vector embedding and store it for fast similarity search. We utilized OpenAI's text-embedding-3-large model to encode text chunks into high-dimensional vectors. This model is part of OpenAI's third-

generation embedding models and produces a numerical representation of text such that semantically similar texts are nearby in the vector space. We chose this model for its strong performance in capturing nuanced semantic relationships, which is important for technical content.

All embeddings were stored in Qdrant, a specialized vector database. Qdrant provides a production-ready service for storing and searching embeddings with efficient similarity search algorithms[4]. We configured Qdrant to use cosine similarity as the distance metric for comparing vectors, since cosine is a standard choice for measuring embedding similarity. The dimensionality of the vectors (embedding size) was automatically determined by the embedding model (we programmatically obtained the vector length from text-embedding-3-large to set up the Qdrant collection). Each vector entry in the database is saved along with its metadata (the document chunk's source info), enabling filtered searches and meaningful results.

The indexing process was implemented as follows (in Python pseudocode for clarity):

```
# Load and preprocess documents

documents = load_documents(KNOWLEDGE_BASE_PATH)        # custom loader for .md and .pdf

chunks = chunk_documents(documents, chunk_size=512)


# Initialize embedding model and vector store

embeddings = OpenAIEmbeddings(model="text-embedding-3-large")

vectorstore = Qdrant.from_documents(

   documents=chunks,

   embedding=embeddings,

   collection_name="vector_collection",

   url=QDRANT_URL, api_key=QDRANT_API_KEY

)
```

This routine reads all files, applies the clause-based splitting and chunking, and then calls Qdrant.from_documents to generate embeddings for each chunk and index them into a Qdrant collection. If the collection already exists from a previous run, the system can reuse it to avoid re-computation, unless a reset is triggered (we included an option to wipe the index and rebuild, useful during development). Storing embeddings in a vector database allows us to retrieve relevant chunks in sub-second time even for large document sets, which is essential for interactive use.

Retrieval and Query Answering Chain: With the vector index in place, our system uses a retrieval-augmented Q&A chain to handle user questions. We built this using LangChain's ConversationalRetrievalChain, which ties together a retriever (vector store) and an LLM for question-answering. The retriever is essentially an interface to Qdrant that, given a new query, searches for similar vectors (i.e. text chunks) and returns the top matching pieces of content. We set up the retriever to return the most relevant chunks for each query, typically a handful of top results ranked by cosine similarity score.

For the LLM, we used OpenAI's GPT models accessed via the API. Here, the models serve as the generative component, tasked with reading the retrieved text chunks and the user's question, and composing a helpful answer. The ConversationalRetrievalChain feeds the query and retrieved context to LLMs in a single prompt, asking it to ground its answer in the provided information. We also integrated a conversation memory buffer (LangChain's ConversationBufferMemory) so that the system can maintain awareness of prior Q&A in a session, enabling follow-up questions to be answered in context. This memory simply accumulates the dialogue history and is used by the chain to prepend relevant history to each new query (to the extent needed to interpret the question).

The chain was initialized in code as shown below:

```
from langchain.chains import ConversationalRetrievalChain

from langchain_openai import ChatOpenAI

from langchain.memory import ConversationBufferMemory


# Prepare the language model and retrieval chain

llm = ChatOpenAI(model_name="gpt-4o", temperature=0)

retriever = vectorstore.as_retriever()  # Qdrant vector store retriever

memory = ConversationBufferMemory(memory_key="chat_history", return_messages=True)


qa_chain = ConversationalRetrievalChain.from_llm(

   llm=llm,

   retriever=retriever,

   memory=memory

)
```

In this configuration, qa_chain will handle the heavy lifting for answering questions: it will automatically use the retriever to get relevant documents from Qdrant and then generate an answer with the llm. When a query comes in, we invoke this chain by passing the user question; the chain returns an object containing both the answer and the source documents it found relevant.

Traceability: The metadata attached during preprocessing is preserved throughout the pipeline, ensuring that every retrieved answer can be referenced back to its source document, section, and path for maximum reliability and transparency.

3.3 Tooling and Frameworks Used

Our system makes use of a comprehensive set of open-source and commercial tools and frameworks, carefully chosen to optimize each level of the RAG pipeline and multiagent orchestration.

3.3.1 Core Frameworks

Langchain: This framework serves as the backbone for our LLM applications. It contains critical components for document loading, text splitting, embedding generation, vectorstore and coordinating the RAG pipeline. LangChain's versatility enables us to smoothly combine numerous tools and models.

CrewAI: This framework is essential for our multi-agent system. CrewAI allows us to define agents with specific roles, goals, and backstories, as well as orchestrate activities amongst them in a systematic workflow. It was useful in implementing the sequential processing and fallback mechanism between both agents. Multi-agent systems enable the division of labor, error mitigation, and layered reasoning which are important for complex technical domains.

3.3.2 Document Management Tools

For document loading, we utilized LangChain's subclasses:

PyMuPDFLoader: For quickly extracting text material from PDF documents, which constitute a large percentage of our structural engineering knowledge base.

TextLoader: For processing plain text files and Markdown documents.

RecursiveCharacterTextSplitter: For intelligent document chunking that respects semantic boundaries during the splitting process.

3.3.3 Vector Store & Embedding

Qdrant: This open-source vector database has been utilized to store and manage our document embeddings. Qdrant offers efficient similarity search

capabilities, which are crucial during RAG's retrieval phase. It is hosted as a cloud service, which guarantees high availability and scalability.

OpenAIEmbeddings (text-embedding-3-large): We used OpenAI's text-embedding-3-large model to create high-quality numerical representations (embeddings) of document chunks and user queries. The quality of these embeddings directly impacts the retrieval accuracy.

ConversationBufferMemory: It preserves chat history and context during multi-turn interactions, which enables follow-up queries and iterative reasoning.

ConversationalRetrievalChain: This chain Integrates retrieval, memory, and LLM-based answer generation to support multi-turn, context-aware interactions.

Environment Setup: For environment variables such as API keys and URLs, a (.env) file was utilized, and document chunks were uniquely identified using uuids.

3.3.4 Interface and Logging

Gradio: We developed a user-friendly chat interface using Gradio, which allows for easy interaction with the Tekla Developer Assistant. Gradio's simplicity allowed for rapid prototype and deployment of the conversational interface.

Logging: Comprehensive logging (logging module) was incorporated across the system to monitor performance, track agent interactions, and help with debugging and problem handling in case of error.

3.3.5 Limitations and Assumptions in Tool Selection

Our tool selection was driven by the need for strong RAG capabilities, efficient vector storage, and adaptable agent orchestration. While these tools give substantial advantages, The system's performance are dependent on the reliability and availability of external services such as OpenAI and Qdrant. This creates a dependency that requires management. The retrieval efficiency varies depending on the loader, splitter, and embedding model employed. While text-embedding-3-large is an effective model, its performance is a necessary assumption for accurate semantic search.

3.4 System Architecture and Component Design

Our architecture is based on the principles of RAG and a multi-agent framework managed by CrewAI. This modular design provides flexibility, scalability, and reliable performance.

3.4.1 Overview of RAG in Context

RAG improves LLM capabilities by providing access to external, current information. Instead of relying solely on pre-trained knowledge, RAG based model gathers relevant documents from a knowledge base before formulating their responses.

This approach considerably reduces the chances of hallucinations and assures that the generated answers derive from provided information. In our scenario, RAG is critical for delivering precise information about certain Finnish annexes that is not covered by the LLM's general training data.

3.4.2 Modular Breakdown of the Framework

The framework consists of various components linked together:

Retriever Module: This module searches and retrieves relevant document chunks from the database. It employs a Qdrant vector storage filled with embeddings produced by OpenAI's text-embedding-3-large model. When a query is received, the retriever performs a similarity search in the vector space to identify the most relevant document chunks.

RAGTool: This is a customised tool, derived from CrewAI's BaseTool, that is specifically designed to handle conversational document queries. This utility contains the whole RAG process, including interactions with the vector store and the LLM to generate replies depending on retrieved content.

The system employs a ConversationalRetrievalChain from LangChain, enhanced with ConversationBufferMemory and InMemoryChatMessageHistory to maintain contextual continuity across multi-turn interactions. This framework assures that responses are contextually consistent, efficient, and traceable by combining retrieval, memory, and conversational reasoning, crucial for complicated structural engineering queries.

Agents: Our system employs a multi-agent approach using CrewAI with two distinct agents to play certain roles [5]:

1. **Query Agent:** This agent is formulated to use the ragtool to obtain document-based answers. It significantly intends to extract accurate information from the knowledge base and provide outputs. In case the tool is unable to find any similar information, this agent is set to halt its operation, signifying that the knowledge base does not contain the answer.

2. **General Agent:** It is a fallback mechanism. It gets called only if the Query Agent cannot get information from within the knowledge base. The general agent relies on its fundamental expertise of Eurocodes and general structural engineering principles to present the response. The function of this agent is significant in order to respond to questions that might be outside the current knowledge base or require

broader expert reasoning irrespective of specific documents. It ensures that a response is always provided, although on the basis of generic expertise rather than specific retrieved documents.

3.4.3 Role and Backstory of Each Agent

Query Agent

This agent will serve as the primary interface for retrieving precise, factual information from the knowledge base of provided structural engineering documents. It uses the RAG pipeline to semantically search and retrieve relevant document chunks from the vectorstore. It would generate clear reference-backed responses rather than speculative answers outside of the knowledge base material. This agent has no delegation rights and operates autonomously within its retrieval scope. If no relevant data is found, this agent will stop its operation and clearly communicate the absence of information.

Role:

“Query Assistant”

Goal:

"Directly return answers obtained from RAG Retrieval Tool without summarizing or altering them."

Backstory:

"You are an expert assistant whose sole purpose is to query the RAG Retrieval Tool and return its output exactly as received, without modification or additional explanation. Never summarize, paraphrase, or alter the RAG tool output. Provide the references exactly as given by the tool."

General Agent

This agent will act as a fallback expert, providing authoritative answers based on pre-trained understanding of Eurocode standards and structural engineering principles. It functions as a specialized agent capable of computing and reasoning based on internal expertise. This agent does not use retrieved document content to assist its own response; however, if the query agent already has an answer, it will simply return the same output without any modification.

Role:

“Structural Engineering Assistant”

Goal:

"A structural engineering expert that answers user questions using its own foundation model-powered knowledge base only if there is no answer from query_agent."

Backstory:

"You are a highly specialized structural engineering expert trained exclusively on the Eurocode standards and fundamental principles of structural design. You serve as the final authority when the Query Assistant fails to find relevant information. Your answers are accurate, standards-compliant, and grounded in Eurocode and best practices, relying solely on your internal expertise. If the query_agent provides an answer, return the same answer without adding a single word."

This allows for a more modular and robust system, where each agent is fine-tuned on its specific domain. Furthermore, developing robust hand-off mechanisms between these agents is crucial, as demonstrated by my system's fallback mechanism.

3.4.4 Task Design Philosophy

The task design philosophy ensures a clear division of work to improve reliability and reduce hallucinations based on agent competence, as well as a strong fallback mechanism inside the multi-agent system. Each task is defined with detailed descriptions and expected outcomes to effectively direct the agents.

Query Task:

The query agent is assigned this task, and it is executed first to retrieve information from the knowledge base to answer and provide a detailed, accurate response with references to source documents. A critical aspect of this task is that if the RAGTool output indicates that no relevant data is found (e.g., "I don't know"), the task is designed to halt, signifying that the knowledge base does not contain the answer. This prevents the Query Agent from fabricating responses.

Description:

f"Use the rag_tool to retrieve detailed information from the knowledge base to exactly answer the question '{query}'. "
"IMPORTANT: The input for this tool MUST be a dictionary with a single key named 'query'. You must directly return the exact output provided by the rag_tool, including all references (Filename, clause, and Page number), without adding, modifying, or summarizing anything. If tool output is I don't know or similar, clearly state exactly: "
f"No relevant data found in the knowledge base for {query}. Please check the query or knowledge base."

Expected Output:

"If rag_tool provides an answer, start with 'According to Finnish Standards,' The exact detailed output returned by the rag_tool, including the explicit references provided by the tool itself (Filename, clause, and Page number) from metadata."

General Task:

This task is assigned to the General Agent and is activated only when the query agent could not find sufficient or relevant information in the knowledge base for its assigned task. The response of query agent would act as a context here. This task embodies the fallback mechanism. It is explicitly forbidden from using retrieved content in this scenario, relying entirely on its internal domain knowledge and reasoning. The explanation should be clear, technically sound, and grounded in recognized engineering practice.

Description:

f"If the query_agent provides an answer, return the same answer without adding a single word. If the query_agent could not find sufficient or relevant information for the query: '{query}', then answer this query based solely on your expertise in structural engineering and Eurocode standards. Do not use retrieved content. Rely entirely on your internal domain knowledge and reasoning. Your explanation should be clear, technically sound, and grounded in recognized engineering practice."

Expected Output:

f"If no relevant information was found for '{query}', provide an expert-level answer including: "
"- A clear, structured explanation. "
"- Reference to applicable Eurocode clauses or principles, if relevant. "
"- Any necessary assumptions or simplifications stated explicitly. "
"- Practical recommendations"
"Ensure the response is informative, even if qualitative."

3.4.5 Integration of Agents via CrewAI

CrewAI coordinates the interaction between the Query Agent and the Structural Engineer Agent in a sequential manner with LLM configurations. The Crew is initialized with both agents and their assigned responsibilities. The Query Task is run first. Its output is then used as context for the General Task. This sequential processing guarantees that the system initially seeks to deliver an accurate, document-based answer from the knowledge base (e.g., Finnish national annexes). If this initial attempt fails, the system will ask the Structural Engineer Agent to respond from its broader and fundamental understanding of Eurocodes. This architecture is consistent with MAS concepts, in which autonomous agents specialize and collaborate to solve complicated

problems efficiently. This hierarchical layered retrieval and expert reasoning approach reduces hallucinations and improves accuracy, while guaranteeing a response for every query. CrewAI's orchestration mirrors human engineering teams' workflows, enhancing robustness and scalability (Mushtaq et al., 2025).

This agent orchestration approach is a simple example of an agentic RAG system: the Query Agent effectively decides whether external knowledge is available, and if not, the next agent steps in to use internal knowledge. It addresses a key limitation of naive RAG pipelines that strictly depend on the vector database. In our case, even if the vector search fails, the user's query is handled gracefully by the LLM. Moreover, by separating the roles, we keep the outputs interpretable – when the knowledge base is used, we clearly present sources, and when the fallback is used, the user implicitly knows the answer is coming from the model's general knowledge (since no reference section is shown).

3.4.6 Flow Diagram of Sequential Processing

CrewAI uses a sequential processing loop to organize the interactions amongst these agents:

1. A user submits a query through the Gradio UI.
2. The query agent responds to the question with the RAGTool by requesting the Qdrant vector storage and, if successful, returns an answer containing references.
3. If the query agent successfully offers an answer, the Structural Engineer Agent receives it as context.
4. The structural engineer agent is ordered to repeat the question agent's response verbatim if one was given. This guarantees that document-based responses are prioritized.
5. If the query agent cannot be able to find pertinent information in the knowledge base, the structural engineer agent will use its pretrained knowledge to provide a response based solely on Eurocodes and general structural engineering knowledge.

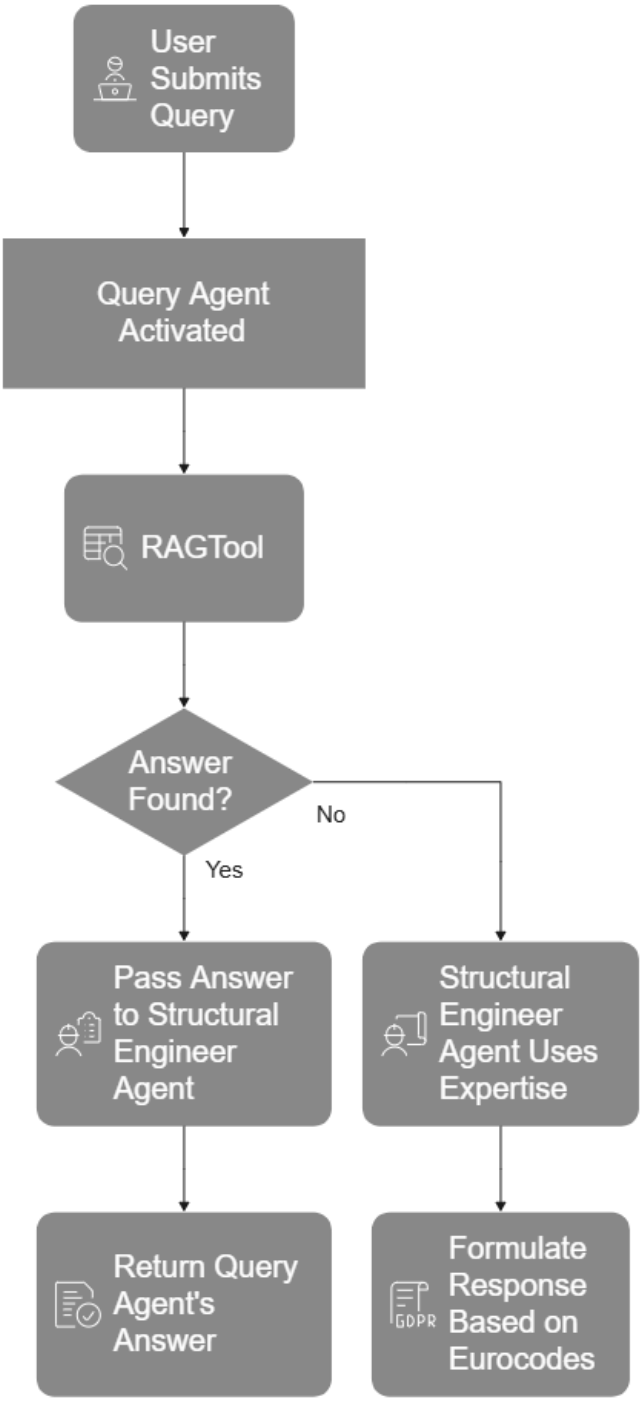

Figure 4 Query Processing and Response Generation

This sequential flow ensures that the system first attempts to provide a precise, document-grounded answer, and only falls back to general expert knowledge when specific information is unavailable. This architecture is

especially useful for answering queries about Finnish national annexes since it prioritizes specialized information while yet providing a strong general engineering perspective.

3.4.7 Scalability Considerations

The architecture is designed with scalability in mind. The modular nature of the RAG components, together with the utilization of cloud-hosted services (Qdrant, OpenAI), allows for easy knowledge base extension and enhanced processing capacity. New documents can be incrementally inserted and processed, and the vector store can be updated with minimal downtime. In the future, the CrewAI framework will permit scaling agent interactions to handle increasingly sophisticated multi-step reasoning tasks.

3.4.8 User Interaction

The entire system was deployed as an interactive chatbot interface to simulate a conversational assistant for structural engineering queries. We used Gradio to create a web-based chat UI, allowing users to type questions and receive answers in real time. The interface code initializes the agents and keeps a chat history so that context is maintained across turns. Each query goes through the pipeline as described, and the answer is displayed to the user with any reference list included. We performed health checks to ensure the OpenAI API and Qdrant vector store were accessible before launching the app. The Gradio ChatInterface was configured with example questions and deployed on a local server for testing.

Throughout development, we focused on making the system's responses transparent and reliable. If the Query Agent found data, the user sees an answer like "According to Finnish Annex, ...". If not, the user gets an answer that is clearly an expert opinion. This way, the methodology ensures that the assistant's answers are either backed by the uploaded documents or, when that's not possible, an educated answer from an expert-like LLM is the next best thing.

By combining modern LLM capabilities with a curated knowledge base and a smart agent orchestration, our system achieves both high accuracy and robustness in answering domain-specific questions. This methodological framework can be extended to other domains where trusted references and complete answers are desired, making it a powerful approach for building intelligent assistants in academic and professional fields.

Figure 5 User Interface

3.5 Evaluation

The evaluation of this thesis was conducted through a structured series of prompt based tests applied to a RAG enabled question answering framework. The system was assessed using a consistent set of 10 structural engineering queries. These queries were designed to test the model's ability to retrieve and interpret technical content from the Finnish Annex of EN 1992. Here are the queries that were tested for our framework:

1. Under what circumstances it is allowed to use partial factor γ_C of 1,35 for ULS for concrete in Finland?
2. When designing for 100 years of service life for a concrete structure, what is the minimum concrete cover c_min,dur for normal outdoor conditions in Finland? Under what circumstances this can be less or more than the default value?
3. In reinforced concrete columns, what is the maximum spacing for loops in columns with dimensions of 480x480 and the main bars of T25 and T20 in each corner?
4. What is the difference between the roles of peripheral and internal ties in reinforced concrete structures? How are these functioning as a whole in the structure and what is the purpose for these ties?
5. What are the most effective ways to anchor reinforcements at support with limited length?
6. What is the maximum height for a reinforced concrete wall of 200 mm of thickness and 20 m of length?
7. For a 30 m tall concrete structure with average of 10 MPa of permanent vertical compression in addition of the self weight, what can be

the maximum total vertical creep and shrinkage of the structure from 28 days of age to end of service?

8. What are possible failure modes for a reinforced concrete corbel, and what are the design checks required for these failure modes in EN1992-1-1?
9. Which checks should I do when designing for the punching shear of a ground-supported reinforced concrete slab without shear reinforcement?
10. What are the loading states to consider when designing for post-tensioned concrete beams?

Each prompt was constructed to target a specific clause, table, or engineering concept from the annex. The goal was to test both the model's retrieval capabilities and its ability to synthesize information into a correct and concise engineering answer.

The prompts were executed in a controlled environment where the models could access the Finnish Annex as the primary source through a retriever component. If the query agent could not generate a response, the fallback mechanism activated the general agent, which used the model's own pretraining knowledge. Only responses from the query agent were used for evaluation.

3.5.1 Hyperparameters

Two main hyperparameters were manipulated to assess the system's performance. The system was evaluated using three distinct language models: GPT-4o, GPT-4-turbo, and GPT-4o-mini, each tested under two temperature settings: 0.0 and 0.7.

LLM Agent

The LLM agent refers to the underlying model responsible for generating the answer based on the context retrieved by the RAG pipeline. Three models were included in this evaluation:

GPT-4o: This is the latest and most advanced model from OpenAI, designed to offer robust performance across a wide range of tasks. Due to its expansive training corpus and enhanced architectural depth, it is especially effective in high-precision tasks such as legal reasoning or engineering compliance. It is, however, computationally demanding.

GPT-4o-mini: A smaller and more efficient version of GPT-4o, optimized for cost and response time. It is particularly useful in applications where affordability, speed, and interpretability are prioritized over deep contextual reasoning. Despite its smaller size, it exhibited a more conservative behavior in our experiments, preferring to avoid answers when insufficient contextual support was found.

GPT-4-turbo: Positioned between GPT-4o and GPT-4o-mini in terms of capabilities and cost, GPT-4-turbo provides balanced performance. In our results, it showed high coverage across all queries, answering every prompt without falling back on general (non-contextual) knowledge, though sometimes at the cost of reduced robustness.

Temperature

Temperature is a hyperparameter that influences the randomness of the generated text. The temperature range usually lies between 0.0 and 2.0. Lower values highlight high probability tokens. This results in deterministic and predictable outputs. These are best for tasks where precision and reliability matter, such as technical explanations. Higher values, increase the chances of selecting less probable tokens, adding variety and creativity. These values work well for creative writing or brainstorming tasks (Davis et al., 2024). Two distinct settings were used:

Temperature = 0.0: This deterministic setting leads the model to consistently select the most probable next token. It is suitable for tasks requiring precise and traceable outputs, such as answering technical queries based on a constrained dataset.

Temperature = 0.7: This allows for more diversity in output by flattening the token probability distribution. It is useful in exploratory or interpretative tasks. In our testing, it sometimes led to responses that included inferred or expanded reasoning not explicitly stated in the source documents.

3.5.2 Evaluation Strategy

Evaluation was conducted both quantitatively and qualitatively. The quantitative metrics included robustness, accuracy and latency, Each metric was analyzed per model temperature combination to assess consistency, reliability, and fidelity to source material. Model behavior was also documented through qualitative analysis to account for the tone, confidence, verbosity, and tendencies to hallucinate or omit information.

4 Results

This chapter presents the results of different LLM configurations in answering engineering questions based on the Finnish National Annex of EN 1992. The study aims to assess the accuracy, robustness, latency and qualitative behavior of these models, particularly when used in a RAG system designed to support structural engineering tasks. The results are validated using actual clauses from the national annex, and the responses are scored and analyzed to determine the effectiveness of each model.

I tested six different configurations of OpenAI's models:

GPT-4o at temperature 0

GPT-4o at temperature 0.7

GPT-4-turbo at temperature 0

GPT-4-turbo at temperature 0.7

GPT-4o-mini at temperature 0

GPT-4o-mini at temperature 0.7

Each model was asked the same 10 domain-specific prompts. The responses of the query agent were then compared to validated answers from the Finnish National Annex of EN 1992. These annex answers served as the ground truth.

The same 10 prompts were used consistently across all models, and the evaluation did not involve any external knowledge or pretrained LLM facts. All validation was performed strictly using the Finnish Annex.

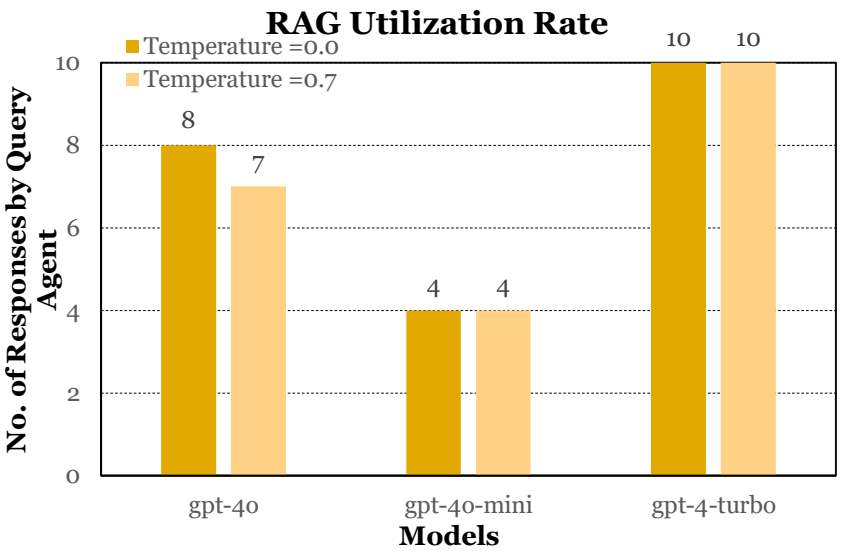

Figure 6 No. of Responses by Query Agent

We found noticeable differences in retrieval utilization rate across the models that can be seen in Fig. 6. GPT-4-turbo provided responses through the query agent for all 10 queries in both temperature settings, suggesting a tendency to always attempt an answer, regardless of how strong the retrieved material was. GPT-4o was slightly more cautious, answering 7 queries at temperature 0 and 6 at temperature 0.7. GPT-4o-mini was the most reserved among the three, producing responses to only 4 out of 10 queries in both settings. These numbers reflect only the responses produced by the query agent itself. In cases where the query agent failed to retrieve sufficient evidence or lacked confidence, a fallback mechanism was triggered. The general agent then stepped in to answer the remaining questions independently, bypassing the retrieval module. This section now proceeds to examine these models more closely in terms of their robustness, accuracy, response latency, and the qualitative integrity of the answers provided via the retrieval-based pipeline.

4.1 Robustness Analysis

We define robustness as the degree to which each model's answers remain grounded in the provided documentation (the Finnish annexes in our RAG pipeline) without introducing unsupported information. A perfectly robust answer uses only retrieved text (with possible omissions if data is missing), whereas a non-robust answer fills gaps with the model's own parametric knowledge or hallucinated details. We assigned each query response a robustness score from 0 to 1, independent of accuracy, reflecting this grounding. To standardize our evaluation, we used scoring scale based on how closely each model's response adhered to the retrieved documents. A score of 1 indicated that the answer was entirely grounded in the provided context with no additions, while 0.75 reflected mostly grounded responses with minor extrapolations. A score of 0.25 suggested minimal reliance on retrieved material, with most of the content generated from the model's internal knowledge. Finally, a score of 0 denoted fully ungrounded answers, typically hallucinated. This rubric enabled a consistent and transparent way to measure how well each model avoided introducing unsupported information. Higher scores indicate answers drawn solely from the documents, while low scores indicate reliance on outside knowledge. Smaller models often simply refrain from answering when information is not found, which boosts robustness (they do not hallucinate), whereas larger models tend to generate an answer regardless, risking the inclusion of unsourced content.

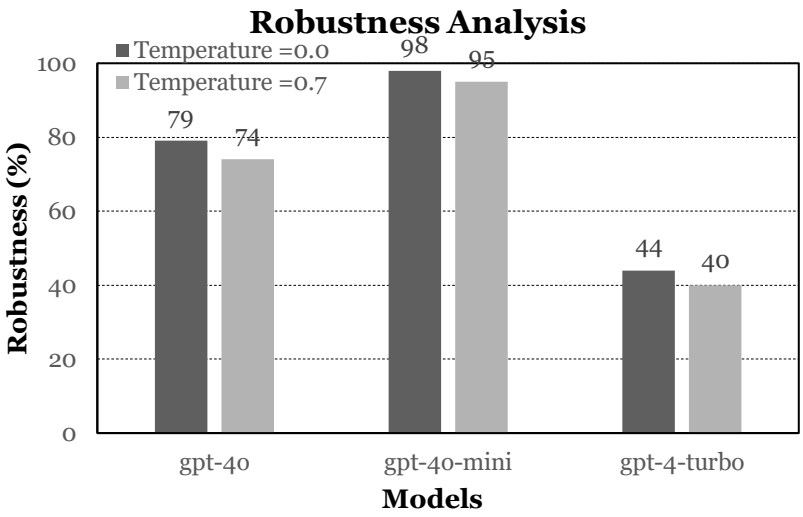

Figure 7 Average Robustness by Model and Temperature

As shown in Fig. 7, GPT-4o-mini exhibited the highest robustness overall, with average scores of about 0.98 at temperature 0 and 0.95 at 0.7. In practical terms, the smallest model almost never hallucinated beyond the provided documents. When it found relevant text, it used it directly, and when it did not, it often gave no answer rather than inventing one. In contrast, GPT-4-turbo was the least robust, with average scores around 0.44 at T=0 and 0.40 T=0.7, indicating that over half of its answer content was not directly supported by the retrieved sources on average. The full-size GPT-4o model was intermediate, scoring roughly 0.79 at T=0 and 0.74 at T=0.7 in robustness. Across all three model variants, using a higher temperature (0.7) caused a slight drop in robustness compared to temperature 0 (deterministic decoding). This trend confirms that smaller models tend to be more robust, likely because their limited parameters discourage them from embellishing answers beyond the given reference text. With fewer internalized facts, GPT-4o-mini stuck to the script or said nothing, yielding pertinent but sometimes incomplete answers. Conversely, the larger models, especially GPT-4-turbo, have a stronger tendency to utilize their vast trained knowledge to fill in missing details, which manifests as lower robustness scores.

From a robustness standpoint, these results reinforce that the query agent pipeline worked best with a smaller, more constrained model and a deterministic decoding strategy. The larger models under stochastic decoding were the least reliable in terms of relevancy. These models frequently injected additional details if not found in the retrieved text. This tendency to hallucinate at higher temperature is well-known, and our data provides concrete evidence of it: e.g. GPT-4-turbo at temp 0.7 had the lowest robustness of any configuration (0.40 average), often producing answers that sounded fluent and confident but were not backed by the reference material.

To evaluate the RAG system's strength, we need to look at how often the models used retrieval and how relevant their answers were. The smaller GPT-4o-

mini used retrieved context to answer only 4 queries through the query agent. However, those answers closely matched the documents. This means that in 6 out of 10 instances, the query agent did not provide a context-based answer because it could not find any relevant snippet. Some might say that in those situations, the model recognized its limits and avoided generating false answers using the query agent.

Conversely, GPT-4-turbo always gave an answer with the query agent, but only about 40% of the content of those answers came from the retrieved documents. The rest relied on the model's internal knowledge or assumptions. This shows that just having a retrieval feature isn't enough; the model also needs to rely on the retrieved information in its answers. Our larger model often viewed the retrieved text as a loose reference or ignored it for its existing knowledge. In real-world use, this behavior could be problematic. The model might give an answer that seems convincing but isn't actually backed by the evidence provided.

4.2 Accuracy Analysis

Accuracy is evaluated as the correctness and completeness of the answer content, regardless of whether the information came from the database or the model's own knowledge. Each query response was annotated for accuracy, with full points for a completely correct answer, partial credit for answers that were mostly correct but missed some details, and zero for incorrect answers. We emphasize that accuracy here is judged by engineering correctness and alignment with Eurocode/Finnish annex requirements, not by relevance from our database. An answer can thus be accurate if it states the right rule or value even if it wasn't derived from the provided text, and conversely an answer can be inaccurate or incomplete even if it only used the documents. We present average accuracy per model.

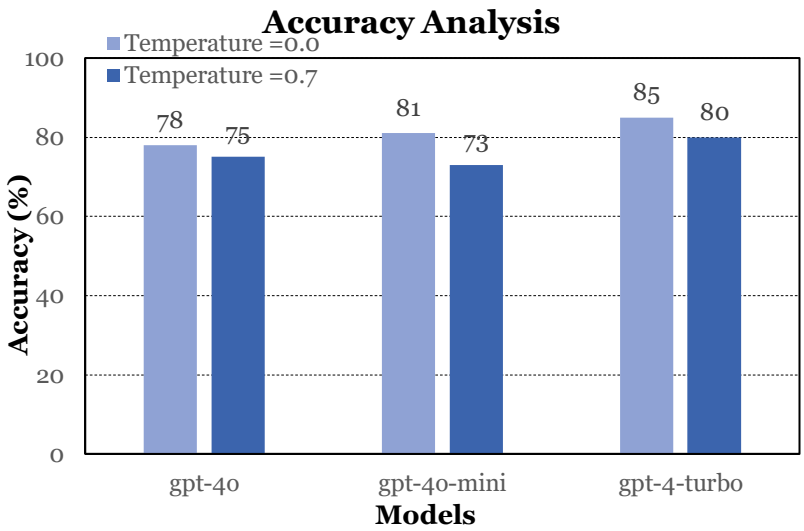

Figure 8 Average Accuracy by Model and Temperature

Each response was evaluated as follows: Score 1: Fully correct and aligns well with the annex. Score 0.75: Mostly correct. Some part is missing or slightly incorrect. Score 0.5: Partially correct. Some part is valid, but important elements are missing or slightly incorrect. Score 0: Incorrect. The bar chart above shows that GPT-4-turbo had the best accuracy overall, with an average score of about 85% at T=0 and 80% at T=0.7. We only evaluated responses which were answered by query agent using RAG pipeline. Out of 10, query agent answered almost every question correctly. Its strong accuracy is attributed to the model's willingness to leverage its broad knowledge even if some answers weren't directly found in the database, GPT-4-turbo often knew the correct engineering principles. For example, on a question about anchoring reinforcements (Query 5), GPT-4-turbo provided a detailed, correct discussion of Eurocode anchorage length requirements and techniques, even though the Finnish annex had no specific guidance. GPT-4o also performed well, with about 78% accuracy at T=0 and 75% at T=0.7. However, the query agent provided 8 responses at T=0 and 7 at T=0.7. Out of those, it answered majority of the questions correctly, though it occasionally left parts unanswered due to its more cautious reliance on documents. Some of GPT-4o's answers were incomplete, which slightly lowered its average accuracy compared to GPT-4-turbo. Meanwhile, GPT-4o-mini trailed in accuracy, around 81% at T=0 dropping to 73% at T=0.7. This might seem surprising given that the query agent often didn't answer many questions at all, so we might expect it to score much lower. The relatively higher average suggests that when GPT-4o-mini did answer, it tended to get it right. In fact, query agent produced fully correct answers for a few questions. It largely avoided giving wrong information; its inaccuracy was usually in the form of omissions (e.g., missing some "words" or failing to provide an equation) rather than explicit errors. In scoring, such omissions were often given partial credit rather than outright zero. By contrast, if we consider total questions answered correctly out of 10 when using GPT-4o-mini, query agent answered only 4 queries from the Finnish Annex. The general agent answered the rest, but those responses were not evaluated for the results. Our evaluation is focused on quality of provided answers. This explains why mini's "accuracy" percentage seems higher than one might expect from its answer count.

Temperature had a minor impact on accuracy. All models saw a slight dip at the higher temperature (0.7) in average accuracy. GPT-4o-mini's accuracy drop was most pronounced from ~81% to ~73%, likely because at higher temperature it was more prone to produce fragmentary attempts or lose focus, leading to more missed details. GPT-4-turbo and GPT-4o each dropped by only a few percentage points at T=0.7. In qualitative terms, the correctness of answers did not dramatically worsen with the moderate increase in randomness, the models still largely produced the same factual content, but we noted a few instances where higher temperature answers included slightly deviated interpretations that we marked down. Overall, accuracy relates positively to model capability. GPT-4-turbo, which engaged all queries and used its knowledge, achieved the highest correctness. In contrast, GPT-4o-mini often went silent when unsure and left several questions unanswered, which

lowered its overall score. GPT-4o performed in the middle; it answered most questions and usually got them right, but sometimes only partially. The connection between robustness and accuracy is complicated here. GPT-4-turbo's less robust approach by adding outside information actually helped it improve accuracy on questions with gaps. On the other hand, GPT-4o-mini's ultra-robust strategy, which never went beyond the provided text, led to failures in answering irrelevant queries. This approach maintains robustness but does not earn points for accuracy. We will discuss this trade-off further in the integrated analysis.

4.3 Processing Time

We also examine the processing time performance of three different language models integrated into a query agent pipeline. It is the total time taken to produce a final answer for each query by the query agent only. In some cases, the specialized retrieval-based "Query agent" could not handle a query, causing the system to fall back to a general agent. We treat those instances as *missing values* for the Query agent's performance, since the query agent effectively skipped those prompts.

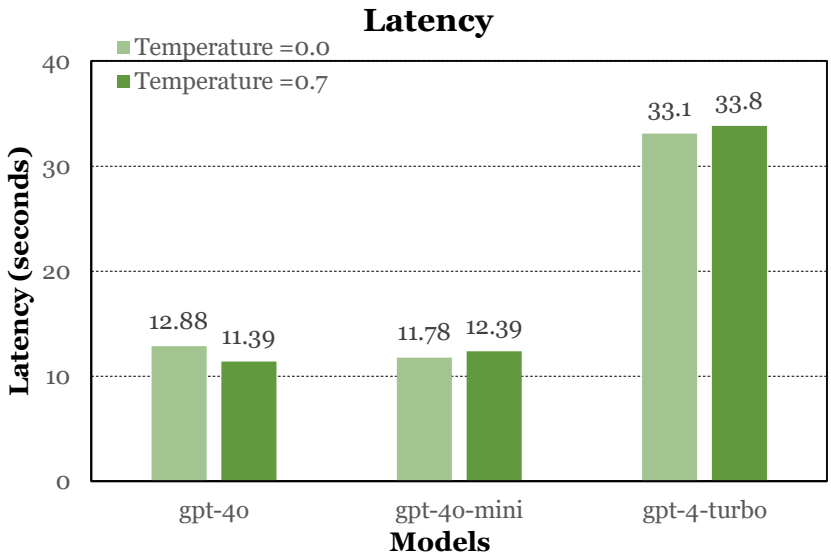

Figure 9 Average Processing Time by Model and Temperature

From the chart, it is immediately clear that gpt-4-turbo is significantly slower than the other models on average. Regardless of temperature, gpt-4-turbo took around 33–34 seconds per query on average. In contrast, gpt-4o and gpt-4o-mini both achieved much lower average latencies in the range of 11–13 seconds. This means that the larger gpt-4-turbo model is roughly 2.5–3 times slower than the lighter models in this pipeline. This consistent lag is not surprising because gpt-4-turbo is presumably a much larger and more complex model. Each inference step takes more computation. This makes sense considering GPT-4-turbo's greater reasoning and longer answers. Its responses were often more elaborate, which required more tokens to be

generated and thus more time. We also observed additional nuances on examining each query in detail. GPT-4-turbo's latency was not only high on average but also highly variable. Its response time fluctuated depending on the question. For example, at T=0, GPT-4-turbo took nearly 54 seconds on Query 5 which is one of the longest answers it gave, involving a detailed explanation of anchorage lengths, whereas it answered Query 1 in about 12 seconds. This variability likely corresponds to the length or complexity of the answers. On the other hand, gpt-4o and gpt-4o-mini stayed in a narrow band, generally between 8 and 18 seconds per query. This indicates a more predictable performance, possibly because the models' answers were concise and they either found the info quickly or gave up without lengthy discursions.

The temperature setting (0 vs 0.7) had only a minor effect on average latency. For gpt-4-turbo, the average times at temperature 0 and 0.7 are virtually the same around 33 seconds. The smaller models show a slight difference, gpt-4o averaged ~12.9sec at T=0 and ~11.4sec at T=0.7, while gpt-4o-mini averaged ~11.8sec at T=0 and ~12.4sec at T=0.7sec. The negligible change in the bar heights between temperatures suggests that temperature does not strongly impact the overall throughput of the models in this pipeline. It's worth noting that the averages for gpt-4o and gpt-4o-mini were calculated over slightly different numbers of queries since the query agent didn't answer all 10 prompts for those models.

4.4 Qualitative Behavior

Beyond the numerical metrics, we qualitatively assessed each model's consistency, reasoning depth, and approach to the tasks. We annotated each response with notes about how the model derived its answer, whether it quoted or extracted from documents correctly, hallucinated or introduced unsupported content, or over-extended the answer beyond what was asked or provided. We also noted the level of detail and any notable reasoning steps. Here we synthesize those observations:

GPT-4o-mini: The mini model was consistent in its cautiousness. It often produced either a concise answer drawn verbatim from a document snippet or no answer at all. For example, for a question asking for a specific provision like query 3 about loop spacing, GPT-4o-mini gave the whole correct answer by directly quoting the required 300 mm spacing from the text, with no embellishment.

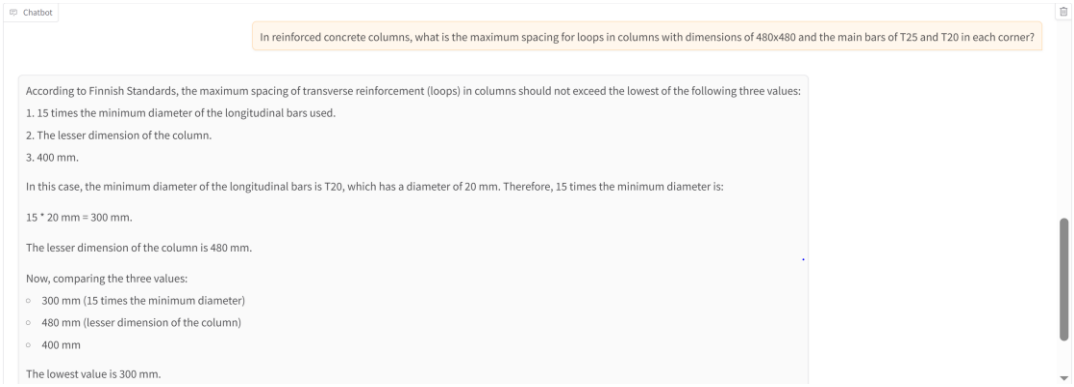

Figure 10 Query 3 Response by gpt-4o-mini

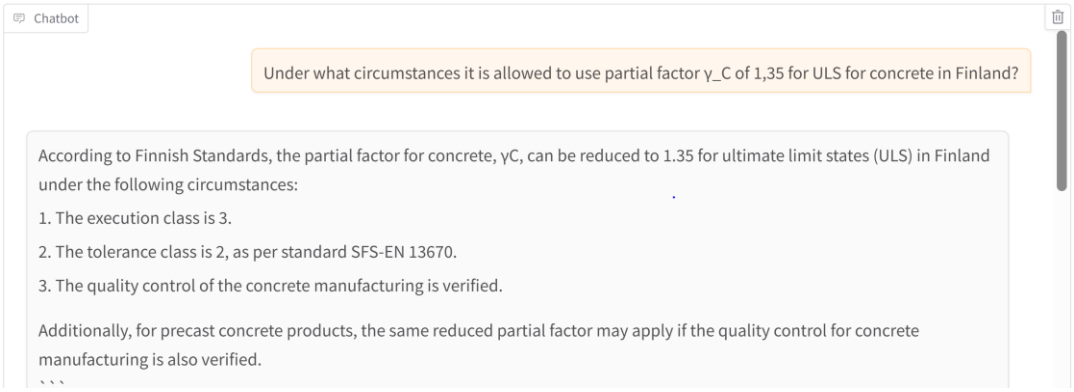

Figure 11 Query 1 Response by gpt-4o-mini

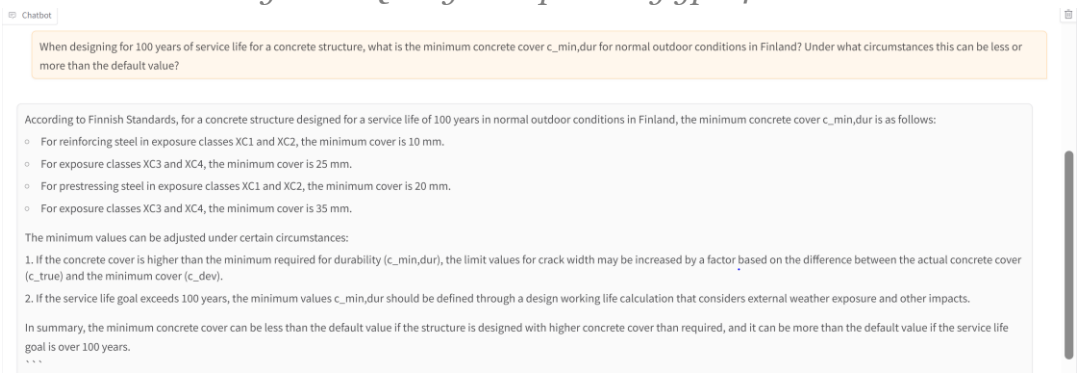

Figure 12 Query 2 Response by gpt-4o-mini

However, when questions required combining information or doing an interpretation, GPT-4o-mini struggled. It frequently missed details, for instance: in case of query 1 and 2, it answered but missed some information from the relevant clause as shown in Fig. 11 and 12.

In cases where data was not explicitly in the documents, GPT-4o-mini simply had no substantive answer. This is evident in the many “No response from query agent” entries for mini, followed often by generic statement from the general agent. Hallucination was virtually absent from GPT-4o-mini’s outputs. It never confidently stated a fact that wasn’t in the sources, at most it would give an incomplete answer or pose that it cannot find info. One downside was that mini sometimes failed to contextualize or reason beyond copy-

pasting. It provided correct values or clauses but without deeper explanation or connection between concepts. Its consistency in style was high which was always brief and, but the depth of reasoning was shallow. In summary, GPT-4o-mini was reliable in not hallucinating, but it fell short when information was not directly available, leading to unanswered questions or answers lacking important pieces (like an equation or justification).

GPT-4o: GPT-4o demonstrated a good balance of using provided information and general knowledge. It generally tried to use the Finnish annex text first. Many of its answers start by quoting or paraphrasing the exact clauses.

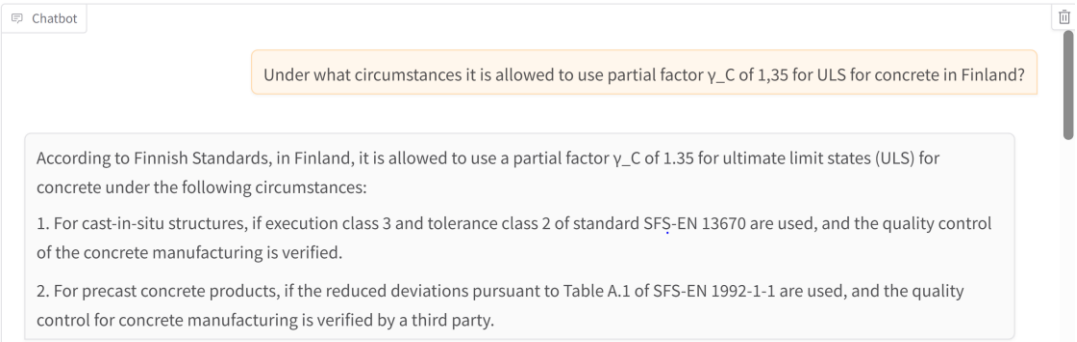

Figure 13 Query 1 Response by gpt-4o

As shown in Fig. 13, Query 1 response about partial factor, GPT-4o retrieved the Finnish National Annex clause and listed the conditions verbatim, which was a correct and sourced answer. It would correctly interpret the documents and sometimes go a step further to explain implications.

GPT-4o sometimes blended its own engineering knowledge into the answer, a mild hallucination in the context of sourcing, but done to increase answer usefulness. Generally, these additions were correct, but they reduced robustness. When information was missing, GPT-4o's fallback (General Agent) often stepped in with a thoughtful general answer. Hallucination in the sense of incorrect facts was very rare for GPT-4o. We did not catch it stating wrong code provisions or impossible values. In some cases, query agent provided correct Eurocode guidance that wasn't in the Finnish annex. In terms of consistency, GPT-4o was highly consistent in language and tone, answers were delivered in a formal, explanatory style and it maintained context well. Reasoning depth for GPT-4o was strong, it was capable of performing intermediate calculations or at least describing them. Sometimes, it refrained from final numerical calculations if data was missing.

GPT-4-turbo: The turbo model's behavior was marked by high verbosity, confidence, and completeness. It consistently produced the longest answers, often merging information from the annex with its own elaborate explanations. In every case, query agent answered the question fully, even if that meant inventing supporting details. For example, for Query 9 which asked which checks to do when something is not explicitly given in the annex. GPT-

4-turbo provided an answer solely from own knowledge and listed out checks and noting which annex clauses might apply, even though the annex itself had no direct list which can be seen in Fig. 14. This was essentially a hallucinated answer, but it was delivered in a very convincing and coherent manner.

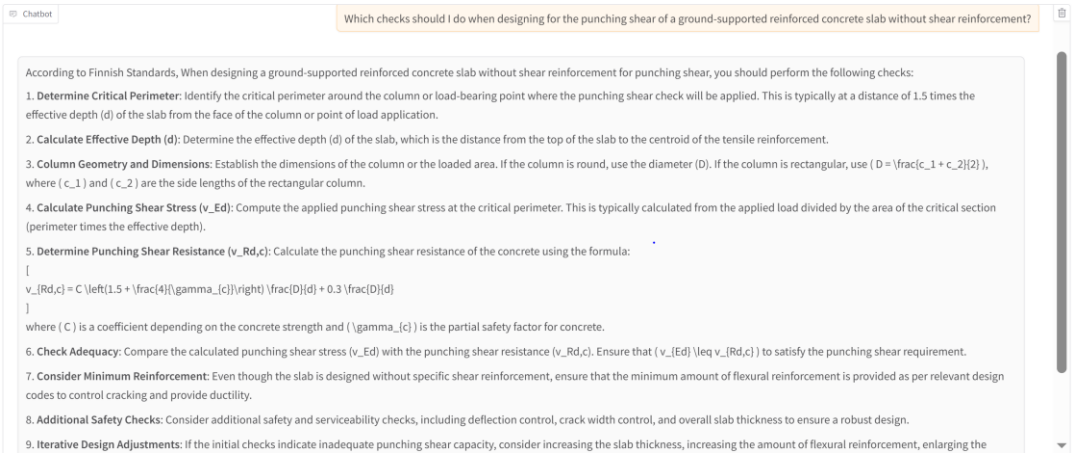

Figure 14 Query 9 Response by gpt-4-turbo

GPT-4-turbo provided an answer solely from own knowledge and listed out checks and noting which annex clauses might apply, even though the annex itself had no direct list. This was essentially a hallucinated answer, but it was delivered in a very convincing and coherent manner. In our evaluation, many of GPT-4-turbo's hallucinations were actually correct or at least reasonable which were drawn from Eurocode general knowledge, hence its high accuracy. The depth of reasoning shown by GPT-4-turbo was impressive. It would not only state what the code says (when it found something) but also add background context, rationale, and even practical recommendations. Such behavior is beneficial for a comprehensive answer but can be risky for accuracy if the model's added content is wrong. GPT-4-turbo maintained a very explanatory and thorough tone throughout, which is consistent, but it sometimes included information not asked in the question. This is a very cautious and detailed style which can be helpful. In terms of staying on track, query agent never outright refused or failed, it always produced something. Overall, GPT-4-turbo's qualitative profile is a model that will always give an answer, a very thorough one by pulling from any relevant knowledge it has, whether or not that knowledge came from the provided documents. It exhibits the classic large-model behavior of hallucinating details confidently, though in our domain-specific case those hallucinations were often on target. Its reasoning was the deepest of the three models.

5 Discussion

This study explored the integration of Retrieval-Augmented Generation (RAG) systems within multi-agent frameworks for structural engineering applications. The discussion builds on the findings and analysis presented in earlier chapters and reflects on how this research aligns with or deviates from prior work in the field. It also aims to interpret the implications of the results and establish a path forward through detailed consideration of the study's contributions, limitations, and future research directions.

The use of RAG in structural engineering contexts has shown promising potential in facilitating more informed responses to domain-specific queries. This thesis implemented a custom RAG pipeline, which was integrated with multiple agents to simulate collaborative problem-solving in a digital engineering environment. The experiment measured model performance in terms of latency, response accuracy, and robustness across a range of temperature settings and prompt variations. The results confirmed that the system could consistently retrieve relevant context and respond with a reasonable degree of accuracy, particularly when prompted using the custom templates designed for the agents.

The system was evaluated across three dimensions: robustness, accuracy, and latency. The results show that the RAG-enhanced agents consistently outperformed the baseline LLMs, particularly when operating under low-temperature settings. In robustness analysis, smaller models such as GPT-4o-mini, when combined with strict retrieval constraints, produced answers that strictly adhered to the input documents. However, these models occasionally left questions partially answered when relevant passages could not be retrieved with high confidence. On the other hand, larger models like GPT-4o and GPT-4 performed well in terms of completeness and interpretability but introduced unsupported details when operating without retrieval constraints. This tendency of larger models to rely on internal knowledge rather than the retrieved context can be problematic in professional settings where traceability and justification of answers are important. Although these models demonstrated high accuracy in many cases, this accuracy was not always linked to the retrieved documents. Thus, while correctness was achieved, the source of the information was not always verifiable within the context provided. This calls for caution in applications requiring evidence-backed decision-making, particularly in regulatory environments.

The findings also suggest that temperature settings had only a moderate influence on model behavior. Across all models, increasing temperature led to a slight decrease in robustness but did not meaningfully impact average accuracy. This outcome reinforces the idea that randomness in generation may introduce variability in style or detail, but the overall reliability of responses remains tied more closely to model size and retrieval use than to temperature alone. However, specific cases showed that higher temperatures could lead

to overly verbose or speculative outputs, which may increase latency without improving relevance.

Latency differences across models were substantial. GPT-4-turbo exhibited the longest response times, especially on complex queries. This latency was partly due to its tendency to provide elaborate explanations, often including additional context that was not explicitly retrieved. While detailed responses can be valuable, the time cost associated with them may not be acceptable in time-sensitive environments. The faster performance of GPT-4o and GPT-4o-mini highlights their suitability for real-time or interactive applications, where response time is a critical factor. The latency results confirm the expected trade-off between performance and speed. Smaller models exhibited faster response times, making them suitable for interactive workflows or mobile applications. Conversely, larger models incurred longer inference times, which may limit their practical use in scenarios requiring real-time feedback. Nonetheless, the qualitative analysis suggests that these larger models, when guided by retrieved context, generate more nuanced and informative explanations, making them suitable for expert use cases.

The qualitative assessment demonstrated that the fallback mechanism implemented through the second agent provided resilience against failures in the retrieval pipeline. This confirms the hypothesis that a multi-agent setup reduces the likelihood of unanswered queries and increases coverage in domain-specific reasoning tasks. Importantly, agentic decomposition also aligns well with the cognitive tasks engineers perform—verifying clauses, retrieving numeric parameters, and reasoning across multiple contexts.

The primary contribution of this thesis lies in demonstrating that domain-aware RAG pipelines can enhance the decision-making capabilities of AI agents in structural engineering. By incorporating domain documents, structural design manuals, and performance benchmarks into the retrieval corpus, the model's output was enriched with relevant and grounded information. Furthermore, the multi-agent system allowed for decomposition of complex tasks, enabling specialized agents to handle subtasks based on their distinct roles. This modularity aligned well with the principles of engineering workflows where various specialists contribute to a single cohesive outcome.

Despite these contributions, the experiments revealed several challenges. While the retrieval component of the pipeline generally performed well, there were consistent issues in extracting precise mathematical content, including equations and formulas. This affected the model's ability to provide reliable quantitative answers. Additionally, although most textual references were retrieved accurately, there were instances where the system either omitted citations or provided incomplete attributions. These limitations suggest that current language models still face constraints when dealing with domain-specific syntax and citation practices common in engineering literature.

Moreover, while the agents were designed with specialized prompts and roles, their capabilities remained bounded by the underlying model's

general-purpose nature. In situations that required deep structural analysis, the agents occasionally generated plausible but technically incorrect responses. This indicates a need for further adaptation and specialization of the models.

From a broader perspective, the thesis advances the role of explainable and collaborative AI systems in engineering. The multi-agent architecture demonstrated the value of task decomposition and parallel reasoning. When agents were allowed to respond independently and a summarizer was used to unify their outputs, the combined response quality improved. This suggests that agent-based models can be more effective when they are both role-aware and allowed to function autonomously before integrating their insights.

5.1 Study Limitations

Despite the strengths of the proposed system, several limitations must be acknowledged. A prominent issue was poor equation extraction from the annex. Many structural code provisions include formulas. The Finnish National Annex documents often present these equations either in LaTeX notation or as embedded images. Our vector database relied on plain text, and the OCR/text extraction step sometimes failed to capture the mathematical expressions correctly. As a result, when a query asked for a specific equation or value that was presented only in a formula, the retrieval agent often struggled to give correct response.

Another limitation was incomplete referencing in the answers. Ideally, an answer would not only state the rule but also pinpoint exactly which clause of the National Annex it came from. Our Agent 1 (Query Agent) was configured to output answers based strictly on the retrieved text, which we chunked from the documents. While this ensures the content is correct, the answer sometimes would read as a standalone statement without explicitly mentioning "according to Clause X of the Finnish National Annex…". The context of the clause might not have been included in the retrieved chunk, so unless the user recognizes it, they have to trust that it's from the annex. From a usability perspective, this is a shortcoming. Engineers would prefer a clear citation of the code clause for confidence. In our current setup, the system stores the sources references in the vector database correctly but unable to integrate that into the answer text in a natural way.

Furthermore, the scope of the domain-specific knowledge was constrained to the Finnish National Annex of Eurocode 2. While this was intentional to allow for focused testing, it limits generalizability to other design domains or code frameworks. The same architecture may need considerable retraining or fine-tuning to accommodate different national annexes or code families such as steel (Eurocode 3), timber (Eurocode 5), etc. Also, the study was confined to three models from the GPT-4 family. Though these models represent a meaningful range in scale and capability, they do not encompass the full diversity of LLM architectures and training strategies available. Future

studies should include models from different providers or open-source alternatives to determine whether similar trade-offs in robustness, accuracy, and latency apply more generally.

Lastly, the fallback agent uses a static model which may be trained on general data that is not updated to the most recent code revisions. Therefore, even when syntactically valid, responses from Agent 2 may become outdated over time, introducing risks in safety-critical design decisions.

5.2 Future Recommendations

Several directions can be pursued to extend this work.

1. Expansion to Other Structural Domains: Future iterations of this system should incorporate additional Eurocode annexes such as EC3 and EC5, allowing evaluation across multiple material systems. This would test the generality of the proposed RAG-based framework and inform its scalability.

2. Expanded Agent Roles: The current setup included a limited number of agent roles. Expanding the system to include more task-specific agents such as optimization agents, code compliance agents, or simulation agents could enhance the pipeline's problem-solving ability. Each agent could be fine-tuned for its respective domain to improve contextual precision.

3. Software Integration: Incorporating the RAG-agent pipeline into design software environments such as RFEM, Tekla, or STAAD Pro would allow for practical, in-context assistance. Such integration would also support real-time feedback and increase adoption by practitioners.

4. Inclusion of Multimodal Inputs: Engineering practice is inherently multimodal. Incorporating visual understanding agents that can interpret structural diagrams, CAD files, or reinforcement layouts would enhance the system's utility and realism.

5. Model Adaptation via Fine-Tuning: Fine-tuning the base LLMs with a dataset of annotated structural design queries and responses may enhance response fidelity, especially for ambiguous or multi-step engineering questions. This could also reduce hallucination rates in fallback agent outputs.

6. Human-in-the-Loop Feedback Loops: A feedback interface where structural engineers can rate or flag responses would help improve system performance over time. Reinforcement learning with human feedback (RLHF) could be explored to fine-tune the generation strategy based on expert guidance.

7. Real-World Pilot Studies with Industry: Collaboration with design firms or code regulatory bodies could lead to pilot implementations of the tool in real projects. Observational studies could yield insight into user trust, failure cases, and workflow integration barriers. Industry partners could also integrate their dynamic and secure datasets directly into the RAG pipeline in a controlled environment. This approach would address security concerns for organisations that do not wish to allow general-purpose models to be trained on their proprietary data, while still enabling the benefits of task-specific retrieval and reasoning.

8. Cross-lingual Adaptation: While this study focuses on English-language queries, many national annexes and engineering teams operate in local languages. Extending the system to multilingual pipelines will increase its accessibility and global applicability.

6 Conclusions

This research focused on creating a practical, reliable method to integrate Finnish National Annex knowledge into LLMs. The primary goal was to address the limitations of current Large Language Models (LLMs), which often lack detailed, localized engineering knowledge. By developing a multi-agent system based on Retrieval-Augmented Generation (RAG), the study demonstrated that combining external, authoritative information with general model knowledge significantly improves the accuracy and reliability of responses to structural design queries.

The research involved designing a two-agent architecture. The first agent retrieved specific clauses and information directly from Finnish National Annex documents. When the first agent couldn't find relevant information, the second agent provided answers based on its general understanding of Eurocodes. This design ensured that every response was grounded in authoritative text whenever possible, greatly reducing the chance of incorrect answers and helping users trust the system more.

Testing and validation showed clearly that the RAG approach offered tangible benefits. By retrieving exact text from official documents, the system successfully reduced the likelihood of incorrect or misleading answers. The evaluation indicated that smaller, more controlled LLMs provided strictly accurate responses based directly on retrieved texts, avoiding unnecessary improvisation. Larger models sometimes offered additional helpful details but occasionally included unsupported information. Therefore, choosing the right LLM type is crucial for balancing answer completeness and strict accuracy. The study also revealed several important limitations. The current retrieval system struggled with accurately extracting mathematical equations. Additionally, referencing was incomplete, making it harder for users to verify the source of information directly. These issues pointed toward clear areas for future improvement, such as better OCR techniques and enhanced referencing procedures.

Practically, this research offers valuable insights for structural engineering professionals. By providing quick and accurate answers to design queries, this system helps engineers save time and reduce the risk of mistakes. The visual dashboard and referencing features give clear insights into how design decisions align with national regulations. Furthermore, integrating such a tool directly into structural design software could enhance workflow efficiency and increase transparency in design compliance processes.

This thesis has delivered a proof-of-concept system that significantly enhances an LLM's ability to function in a niche, high-accuracy domain by leveraging external data and multiple agents. We have shown that LLMs need not operate as isolated black boxes when connected to domain-specific knowledge bases and organized into collaborative agents, they become far more powerful and reliable for specialized applications. The work contributes both a tangible tool (the structural design assistant prototype) and broader

insights into human-AI collaboration in Structural Engineering. By adopting such AI assistants, the structural engineering industry can improve efficiency and consistency, all while upholding the rigorous safety standards that protect the public. The bridge between advanced AI and everyday engineering practice is being built, and this thesis has laid some of the foundational planks on that bridge, pointing the way toward a future where checking a design against a hundred-page national annex is as easy as having a conversation with your AI colleague.

References

Abduljaleel, Y. W., Usman, F., Syamsir, A., Albaker, B. M., Najeeb, M. I., Khattab, M. M., & Al-Humairi, S. N. (2025). A Systematic Review on Utilizing Artificial Intelligence in Lateral Resisting Systems of Buildings. *Archives of Computational Methods in Engineering*, 1-68.

Acharya, D. B., Kuppan, K., & Divya, B. (2025). Agentic AI: Autonomous Intelligence for Complex Goals–A Comprehensive Survey. *IEEE access*.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., & Askell, A. (2020). Language models are few-shot learners. *Advances in neural information processing systems*, *33*, 1877-1901.

Cha, Y. J., Choi, W., & Büyüköztürk, O. (2017). Deep learning-based crack damage detection using convolutional neural networks. *Computer-Aided Civil and Infrastructure Engineering*, *32*(5), 361-378.

Cheng, M., Luo, Y., Ouyang, J., Liu, Q., Liu, H., Li, L., Yu, S., Zhang, B., Cao, J., & Ma, J. (2025). A survey on knowledge-oriented retrieval-augmented generation. *arXiv preprint arXiv:2503.10677*.

D'Antonoli, T. A., Stanzione, A., Bluethgen, C., Vernuccio, F., Ugga, L., Klontzas, M. E., Cuocolo, R., Cannella, R., & Koçak, B. (2024). Large language models in radiology: fundamentals, applications, ethical considerations, risks, and future directions. *Diagnostic and Interventional Radiology*, *30*(2), 80.

Davis, J., Van Bulck, L., Durieux, B. N., & Lindvall, C. (2024). The temperature feature of ChatGPT: modifying creativity for clinical research. *JMIR human factors*, *11*(1), e53559.

Dyke, S. J., Liu, X., Zhang, X., & Iturburu, L. (2024). Empowering Engineers by Leveraging AI in Structural Engineering and Monitoring. *Procedia Structural Integrity*, *64*, 21-28.

Fan, W., Ding, Y., Ning, L., Wang, S., Li, H., Yin, D., Chua, T.-S., & Li, Q. (2024). A survey on rag meeting llms: Towards retrieval-augmented large language models. Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining,

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, H., & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, *2*(1).

Ghimire, P., Kim, K., & Acharya, M. (2023). Generative ai in the construction industry: Opportunities & challenges. *arXiv preprint arXiv:2310.04427*.

Göpfert, J., Weinand, J. M., Kuckertz, P., & Stolten, D. (2024). Opportunities for large language models and discourse in engineering design. *Energy and AI*, *17*, 100383.

Gupta, S., Ranjan, R., & Singh, S. N. (2024). A comprehensive survey of retrieval-augmented generation (rag): Evolution, current landscape and future directions. *arXiv preprint arXiv:2410.12837*.

He, C., Yu, B., Liu, M., Guo, L., Tian, L., & Huang, J. (2024). Utilizing large language models to illustrate constraints for construction planning. *Buildings*, *14*(8), 2511.

Hughes, L., Dwivedi, Y. K., Malik, T., Shawosh, M., Albashrawi, M. A., Jeon, I., Dutot, V., Appanderanda, M., Crick, T., & De', R. (2025). AI agents and agentic systems: A multi-expert analysis. *Journal of Computer Information Systems*, 1-29.

Jadhav, Y., & Farimani, A. B. Large language model agent as a mechanical designer, 2024. *URL https://arxiv. org/abs/2404.17525*.

Kampelopoulos, D., Tsanousa, A., Vrochidis, S., & Kompatsiaris, I. (2025). A review of LLMs and their applications in the architecture, engineering and construction industry. *Artificial Intelligence Review*, *58*(8), 250.

Kang, T., & Kang, K. (2024). Earthwork Network Architecture (ENA): Research for Earthwork Quantity Estimation Method Improvement with Large Language Model. *Applied Sciences*, *14*(22), 10517.

Liang, C.-J., Le, T.-H., Ham, Y., Mantha, B. R., Cheng, M. H., & Lin, J. J. (2024). Ethics of artificial intelligence and robotics in the architecture, engineering, and construction industry. *Automation in Construction*, *162*, 105369.

Liao, W., Lu, X., Fei, Y., Gu, Y., & Huang, Y. (2024). Generative AI design for building structures. *Automation in Construction*, *157*, 105187.

Murugesan, S. (2025). The rise of agentic AI: implications, concerns, and the path forward. *IEEE Intelligent Systems*, *40*(2), 8-14.

Mushtaq, A., Naeem, M. R., Ghaznavi, I., Taj, M. I., Hashmi, I., & Qadir, J. (2025). Harnessing Multi-Agent LLMs for Complex Engineering Problem-Solving: A Framework for Senior Design Projects. *arXiv preprint arXiv:2501.01205*.

Naveed, H., Khan, A. U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Akhtar, N., Barnes, N., & Mian, A. (2023). A comprehensive overview of large language models. *ACM Transactions on Intelligent Systems and Technology*.

Nguyen, T.-T., Dang, V.-H., & Pham, T.-T. (2024). Development of an automatic and knowledge-infused framework for structural health

monitoring based on prompt engineering. *Frontiers of Structural and Civil Engineering*, *18*(11), 1752-1774.

Pokhrel, R., & Parajuli, S. (2023). Transforming Structural Engineering: Examining the Opportunities and Risks of ChatGPT and Other Large Language Models. *International Journal on Engineering Technology*, *1*(1), 204-222.

Prabhune, S., & Berndt, D. J. (2024). Deploying Large Language Models With Retrieval Augmented Generation. *arXiv preprint arXiv:2411.11895*.

Qin, S., Guan, H., Liao, W., Gu, Y., Zheng, Z., Xue, H., & Lu, X. (2024). Intelligent design and optimization system for shear wall structures based on large language models and generative artificial intelligence. *Journal of Building Engineering*, *95*, 109996.

Qu, R., Tu, R., & Bao, F. (2024). Is Semantic Chunking Worth the Computational Cost? *arXiv preprint arXiv:2410.13070*.

Raiaan, M. A. K., Mukta, M. S. H., Fatema, K., Fahad, N. M., Sakib, S., Mim, M. M. J., Ahmad, J., Ali, M. E., & Azam, S. (2024). A review on large language models: Architectures, applications, taxonomies, open issues and challenges. *IEEE access*, *12*, 26839-26874.

Ravuru, C., Sakhinana, S. S., & Runkana, V. (2024). Agentic retrieval-augmented generation for time series analysis. *arXiv preprint arXiv:2408.14484*.

Salehi, H., & Burgueño, R. (2018). Emerging artificial intelligence methods in structural engineering. *Engineering structures*, *171*, 170-189.

Sammour, F., Xu, J., Wang, X., Hu, M., & Zhang, Z. (2024). Responsible AI in Construction Safety: Systematic Evaluation of Large Language Models and Prompt Engineering. *arXiv preprint arXiv:2411.08320*.

Sapkota, R., Roumeliotis, K. I., & Karkee, M. (2025). Ai agents vs. agentic ai: A conceptual taxonomy, applications and challenges. *arXiv preprint arXiv:2505.10468*.

Sargiotis, D. (2024). Advancing civil engineering with AI and machine learning: from structural health to sustainable development. *Available at SSRN 4883999*.

Schneider, J. (2025). Generative to agentic ai: Survey, conceptualization, and challenges. *arXiv preprint arXiv:2504.18875*.

Siddharth, L., & Luo, J. (2024). Retrieval augmented generation using engineering design knowledge. *Knowledge-Based Systems*, *303*, 112410.

Singh, A., Ehtesham, A., Kumar, S., & Khoei, T. T. (2025). Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG. *arXiv preprint arXiv:2501.09136*.

Spencer Jr, B. F., Sim, S.-H., Kim, R. E., & Yoon, H. (2025). Advances in artificial intelligence for structural health monitoring: A comprehensive review. *KSCE Journal of Civil Engineering*, *29*(3), 100203.

Thai, H.-T. (2022). Machine learning for structural engineering: A state-of-the-art review. Structures,

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, *30*.

Wu, C., Ding, W., Jin, Q., Jiang, J., Jiang, R., Xiao, Q., Liao, L., & Li, X. (2025). Retrieval augmented generation-driven information retrieval and question answering in construction management. *Advanced Engineering Informatics*, *65*, 103158.

Zahedifar, R., Mirghasemi, S. A., Baghshah, M. S., & Taheri, A. (2025). LLM-Agent-Controller: A Universal Multi-Agent Large Language Model System as a Control Engineer. *arXiv preprint arXiv:2505.19567*.

Zhao, P., Zhang, H., Yu, Q., Wang, Z., Geng, Y., Fu, F., Yang, L., Zhang, W., Jiang, J., & Cui, B. (2024). Retrieval-augmented generation for ai-generated content: A survey. *arXiv preprint arXiv:2402.19473*.